Дисциплина «Проектирование архитектуры ПО»

Кафедра корпоративных информационных систем

# ОТЧЁТ

по практической работе №1

Тема: Развёртывание контейнеризированных сред для нескольких типов баз данных

Выполнил: студент группы БСБО-01-23

Судовский Андрей Леонидович

Москва, 2026

**СОДЕРЖАНИЕ**

1. Цель работы ..... 4

2. Задание ..... 4

3. Среда выполнения ..... 4

4. Состав стенда ..... 4

5. Запуск стенда и проверка доступности ..... 6

5.1. Redis ..... 7

5.2. MongoDB ..... 8

5.3. Neo4j ..... 10

5.4. Elasticsearch ..... 10

5.5. PostgreSQL ..... 11

6. Места хранения данных и журналов ..... 12

7. Задания для самостоятельной работы ..... 15

7.1. Смена паролей во всех конфигурациях ..... 16

7.2. Скрипт инициализации PostgreSQL ..... 18

7.3. MongoDB: вложенные документы и агрегация ..... 19

7.4. Redis: счётчик посещений с TTL ..... 20

7.5. Neo4j: «друзья друзей» ..... 21

7.6. Elasticsearch: полнотекстовый поиск с ранжированием ..... 22

8. Выводы ..... 23

Список использованных источников ..... 24

Приложение А. Файл docker-compose.yml ..... 26

Приложение Б. Смена паролей: selfwork\rotate_passwords.py ..... 29

Приложение В. Скрипты инициализации PostgreSQL: selfwork\initdb ..... 32

## 1. Цель работы

Развернуть в контейнерах Docker пять служб, обеспечивающих различные виды представления хранимых данных: Redis (ключ-значение), MongoDB (документы), Neo4j (граф), Elasticsearch (полнотекстовый поиск) и PostgreSQL (реляционная модель). Для каждой службы выполнить проверку доступности и настроить место хранения данных и информации о состоянии (лог-записей).

## 2. Задание

«Выполнить развертывание контейнеров или установить локально следующие службы, обеспечивающие различные виды представления хранимых данных: Redis, MongoDB, Neo4j, ElasticSearch, PostgreSQL. Выполнить для каждой службы проверку доступности и настройку места хранения данных и информации о состоянии (лог-записи)».

Работа выполнена по методическим указаниям [2] к заданию [1]. Контейнеры и СУБД настроены по официальной документации [3–8], модели данных и области применения СУБД сопоставлены по [9].

## 3. Среда выполнения

Таблица 1 — Среда выполнения работы

| **Компонент** | **Версия / параметры** |
|---|---|
| Операционная система | Windows 10 Pro 22H2 (сборка 19045), 16 ГБ ОЗУ, 8 логических процессоров |
| Docker Desktop | 4.88.1, движок Docker Engine 29.7.2, Docker Compose v5.4.0 |
| Подсистема виртуализации | WSL 2 версии 3.0.1, ядро 6.18.40.1; память виртуальной машины ограничена 6 ГБ (файл `%USERPROFILE%\.wslconfig`, параметр `memory=6GB`) |
| Рабочий каталог | `SAD\practice-db\pr1`: `docker-compose.yml`, `.env`, `logs\`, `check.sh`, `evidence\` |

## 4. Состав стенда

Все пять СУБД и их веб-интерфейсы описаны в одном файле `docker-compose.yml` (вариант «единый docker-compose файл» из раздела 6 методических указаний) и запускаются одной командой `docker compose`

`up -d`. Контейнеры находятся в общей сети проекта `practice01_default`, поэтому веб-интерфейсы обращаются к СУБД по именам служб (`redis`, `mongodb`, `postgres` и т. д.). Учётные данные вынесены из файла описания в файл `.env` рядом с ним.

Таблица 2 — Службы стенда

| **Служба** | **Вид хранения** | **Образ (версия)** | **Порт на хосте** | **Веб-интерфейс** |
|---|---|---|---|---|
| Redis | ключ-значение | `redis:7-alpine` (7.4.11) | 6379 | Redis Commander 0.7.2, :8081 |
| MongoDB | документы | `mongo:7` (7.0.45) | 27017 | Mongo Express 1.0.2, :8082 |
| Neo4j | граф | `neo4j:5-community` (5.26.32) | 7474, 7687 | Neo4j Browser (встроен), :7474 |
| Elasticsearch | полнотекстовый поиск | `elasticsearch:8.16.0` | 9200 | Kibana 8.16.0, :5601 |
| PostgreSQL | реляционная | `postgres:16-alpine` (16.15) | 5433 | pgAdmin 4 (9.18), :8083 |

Отличия от примеров методических указаний и их причины:

– PostgreSQL опубликован на порту хоста **5433**: порт 5432 на рабочем компьютере уже занят локально установленным PostgreSQL. Внутри сети Docker служба по-прежнему слушает 5432, поэтому pgAdmin подключается к `postgres:5432` без изменений.

– Ключ `version` в начале файла не указан: в Docker Compose v2 и новее он устарел и только выводит предупреждение.

– Журналы каждой СУБД пишутся в файлы, а каталоги журналов смонтированы на хост в `pr1\logs\<служба>`; подробнее — раздел 6.

– Для Redis включена персистентность AOF вместе со снимками RDB (`--appendonly yes --save 60 1`): без этого данные Redis после перезапуска восстанавливаются только из последнего снимка.

– Лимит памяти Elasticsearch и Kibana увеличен с 1 до 1,5 ГБ: при лимите 1 ГБ контейнер Elasticsearch занимал около 950 МБ (heap 512 МБ плюс память JVM вне heap) и работал у самого предела.

– Проверка готовности (healthcheck) Neo4j выполняется HTTP-запросом к порту 7474. Вариант через `cypher-shell` при каждом вызове запускал

отдельную JVM и давал кратковременную загрузку процессора до 180 %.

– Сервер «Practice PostgreSQL» регистрируется в pgAdmin автоматически из файла `pgadmin\servers.json`, пароль подключения вводится при первом соединении.

## 5. Запуск стенда и проверка доступности

Образы загружены командой `docker compose pull` (9 образов, около 7,7 ГБ на диске), стенд запущен командой `docker compose up -d`. Для пяти СУБД заданы проверки готовности (healthcheck), поэтому веб-интерфейсы стартуют только после того, как соответствующая СУБД отвечает на запросы (`depends_on: condition: service_healthy`). Рабочая нагрузка стенда — около 2,9 ГБ ОЗУ из 5,65 ГБ, выделенных Docker (рисунок 1).

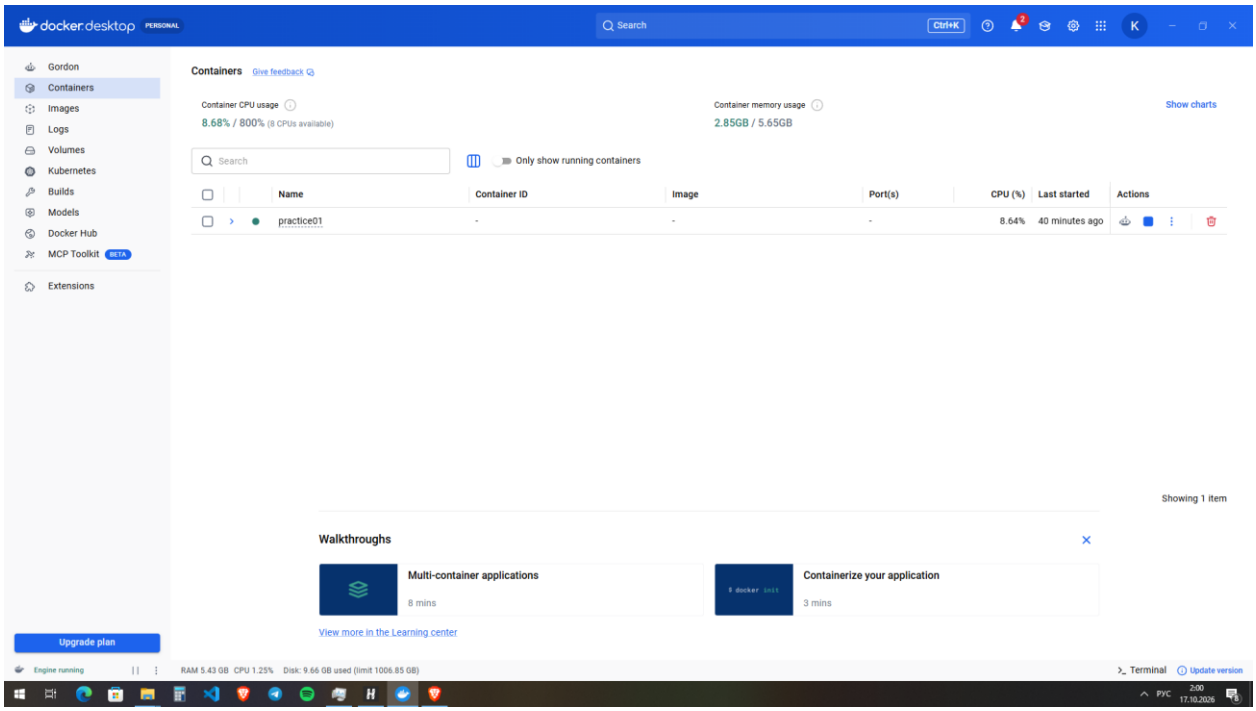



Рисунок 1 — Docker Desktop: проект practice01 запущен

Состояние контейнеров и созданные тома показаны на рисунке 2: у всех пяти СУБД статус `(healthy)`, т. е. проверка готовности проходит.

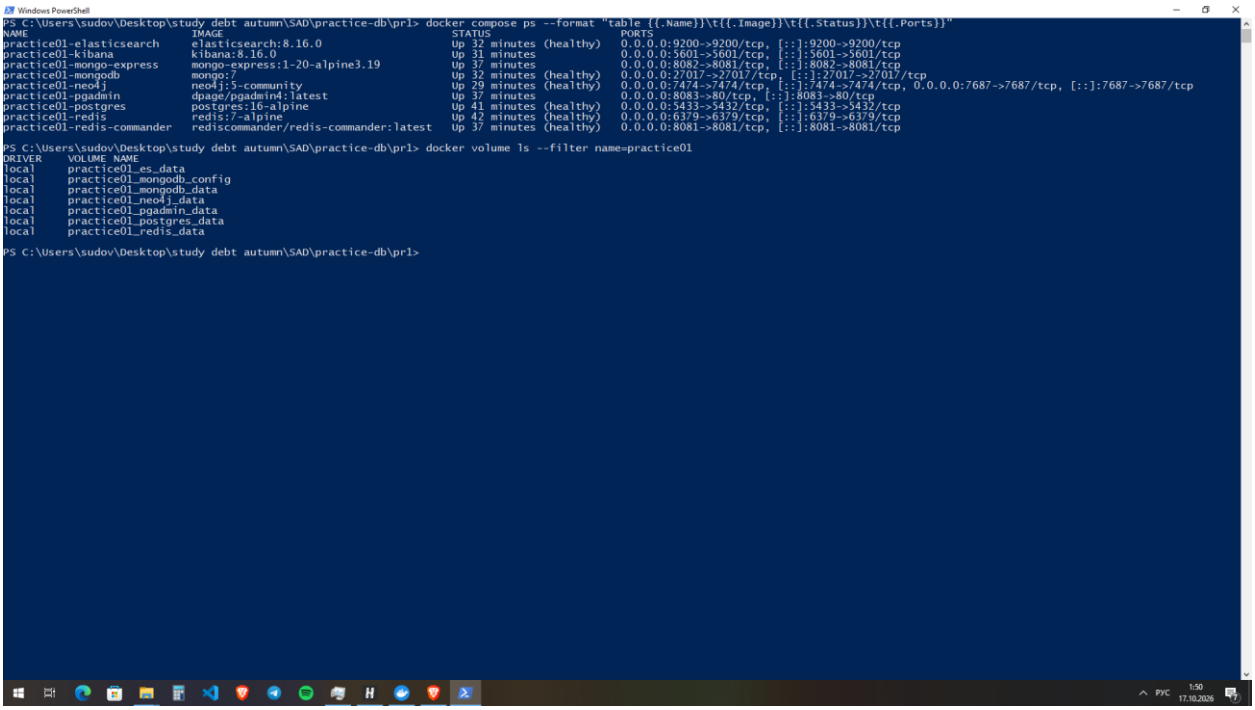


Рисунок 2 — Состояние контейнеров (docker compose ps) и тома проекта

## 5.1. Redis

Проверка выполнена в `redis-cli` внутри контейнера (рисунок 3): команда `PING` возвращает `PONG`; `CONFIG GET dir` показывает каталог данных `/data` (именованный том `practice01_redis_data`), `CONFIG GET appendonly` — включённый журнал AOF, `CONFIG GET logfile` — файл журнала `/var/log/redis/redis.log`. Базовые операции: запись строки `SET`, чтение `GET`, хеш `HSET`/`HGETALL`.

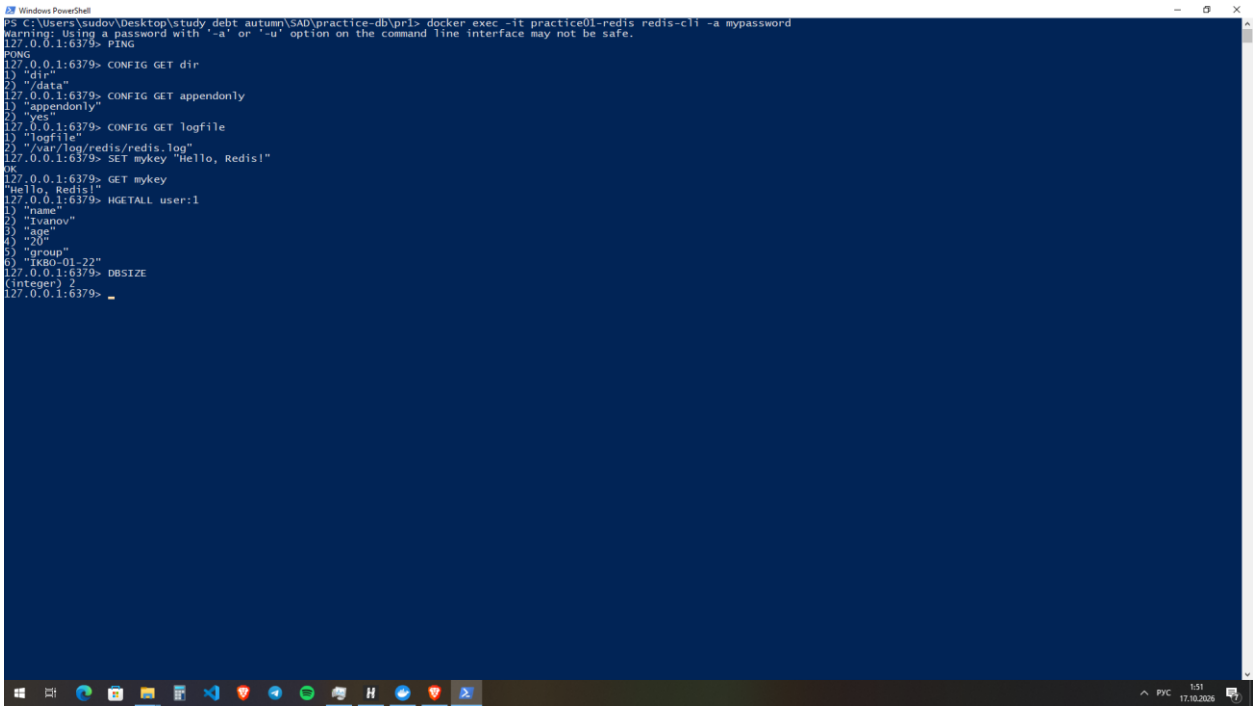



Рисунок 3 — Redis: проверка доступности, каталог данных и журнала, базовые операции

Те же ключи видны в веб-интерфейсе Redis Commander (рисунок 4).

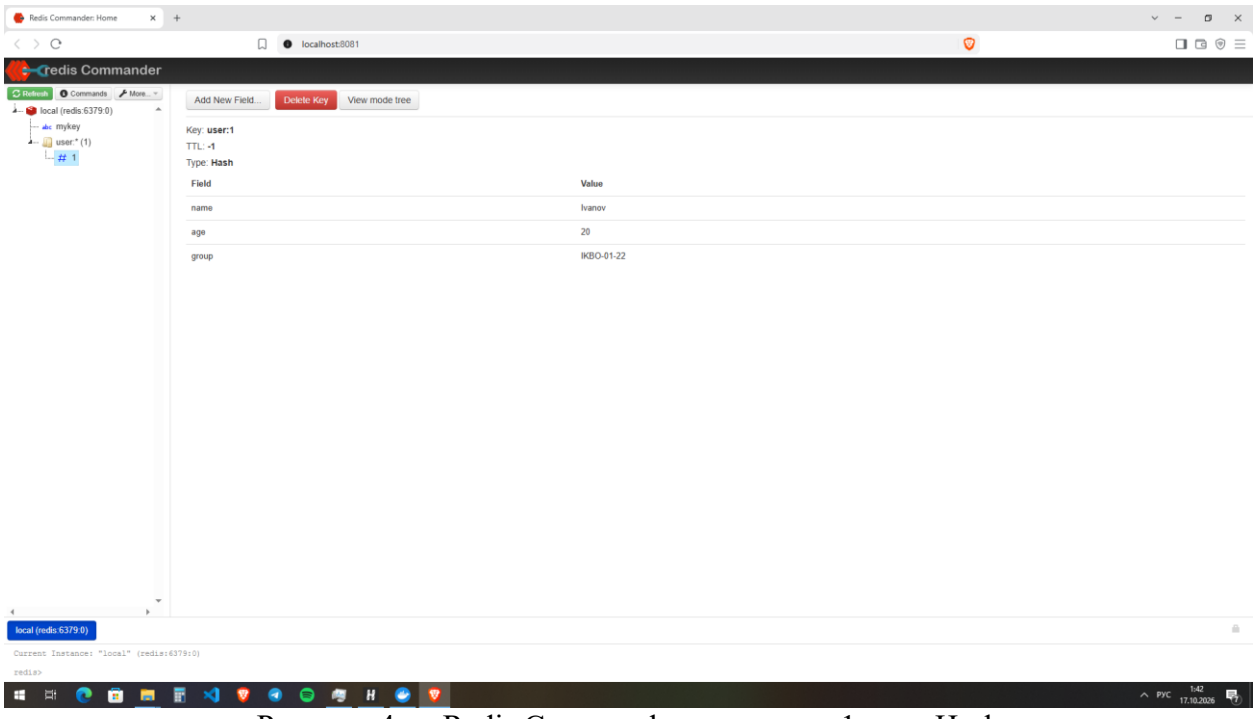



Рисунок 4 — Redis Commander: ключ user:1 типа Hash

## 5.2. MongoDB

В `mongosh` (рисунок 5) проверены версия сервера (7.0.45) и фактические параметры запуска `mongod`: включена авторизация, кеш WiredTiger ограничен 0,5 ГБ, журнал пишется в файл

`/var/log/mongodb/mongod.log` в режиме дозаписи. В базу `practice_db` добавлены три документа коллекции `students`; они же видны в Mongo Express (рисунок 6).

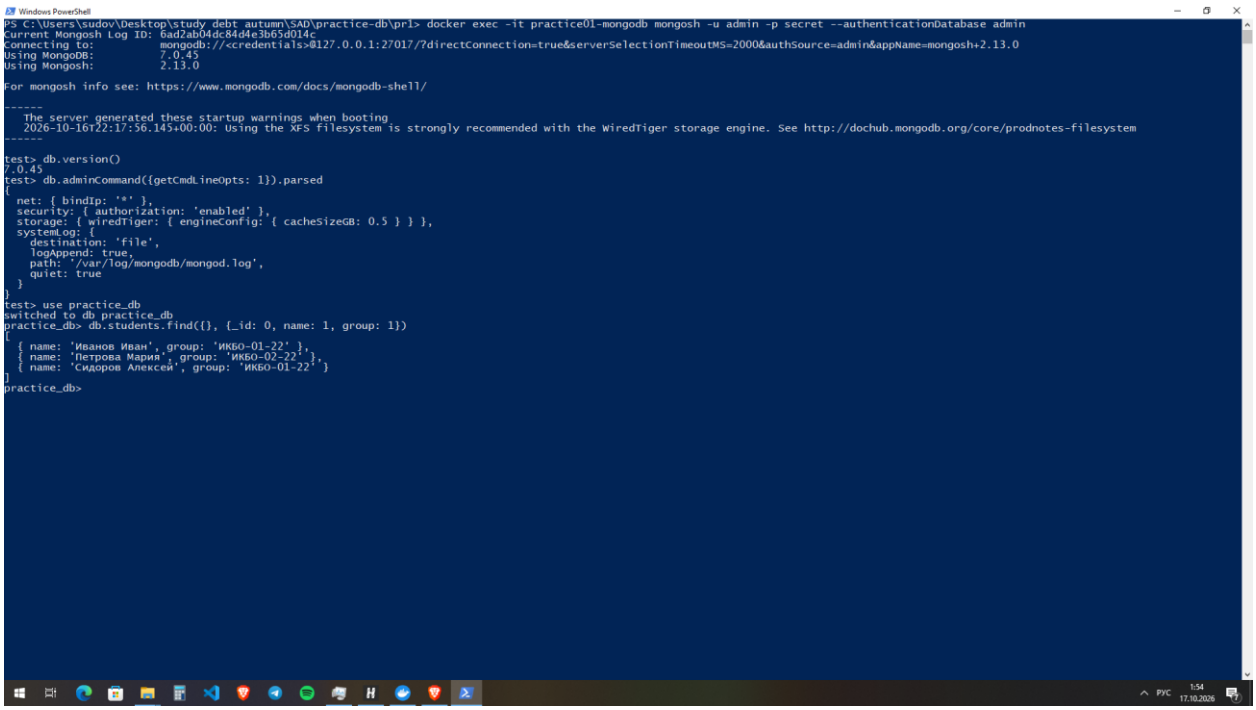



Рисунок 5 — MongoDB: версия, параметры запуска mongod, выборка из коллекции students

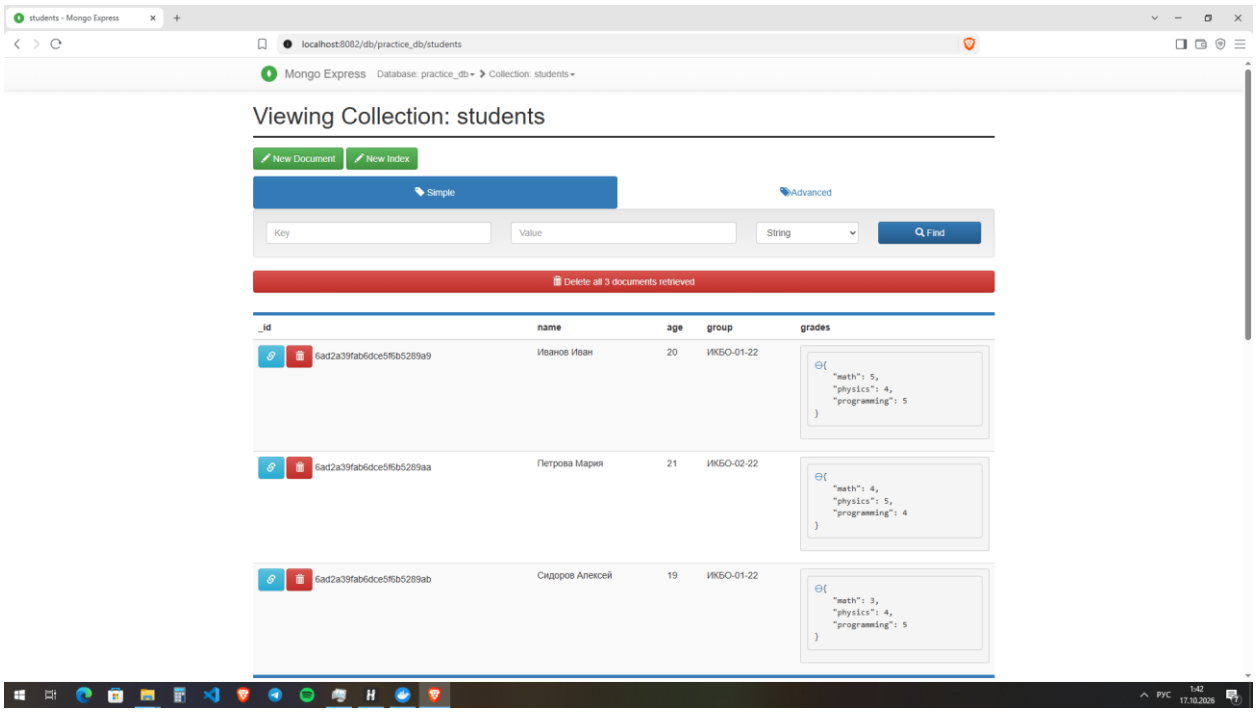



Рисунок 6 — Mongo Express: коллекция students базы practice_db

### 5.3. Neo4j

Доступность проверена через Neo4j Browser по протоколу Bolt (`neo4j://localhost:7687`). Созданы узлы `Student` и `Course` и связи `ENROLLED_IN`; запрос `MATCH (n)-[r]->(m) RETURN n, r, m` выводит граф из четырёх узлов и трёх связей (рисунок 7). Каталоги данных и журналов, по данным `SHOW SETTINGS`: `/var/lib/neo4j/data` (смонтирован как `/data`, том `practice01_neo4j_data`) и `/logs`.

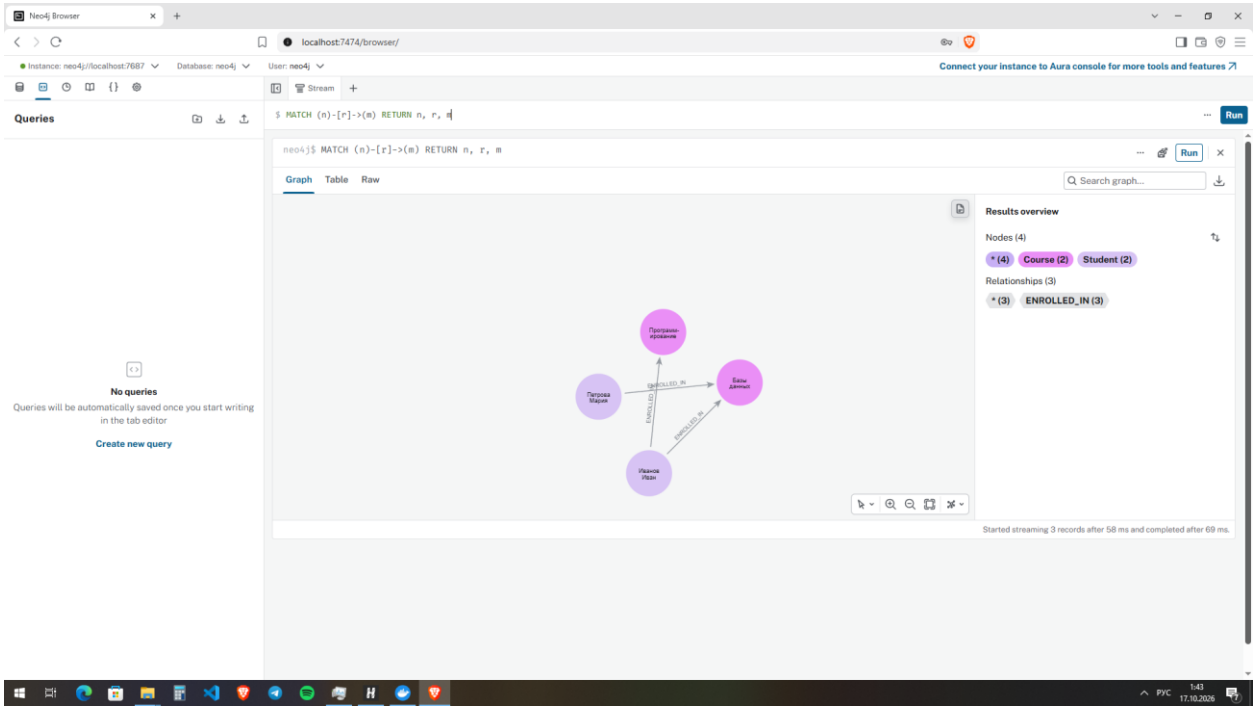



Рисунок 7 — Neo4j Browser: граф студентов и курсов

### 5.4. Elasticsearch

Состояние кластера получено REST-запросом `GET /_cluster/health` (рисунок 8): статус `green`, один узел, все шарды размещены. В индекс `students` с полями `text` и `keyword` добавлены два документа; полнотекстовый запрос `match` по полю `bio` со словом «данных» находит документ, где это слово встречается (рисунок 9, консоль Dev Tools в Kibana).

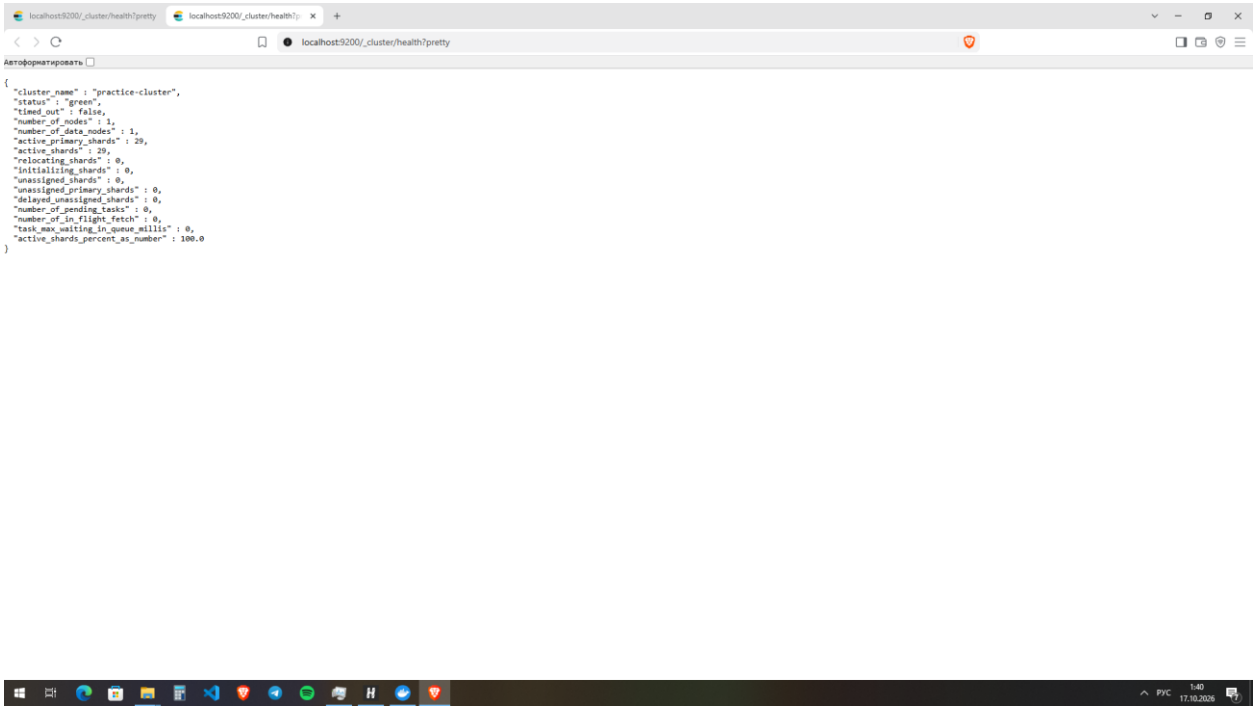



Рисунок 8 — Elasticsearch: состояние кластера

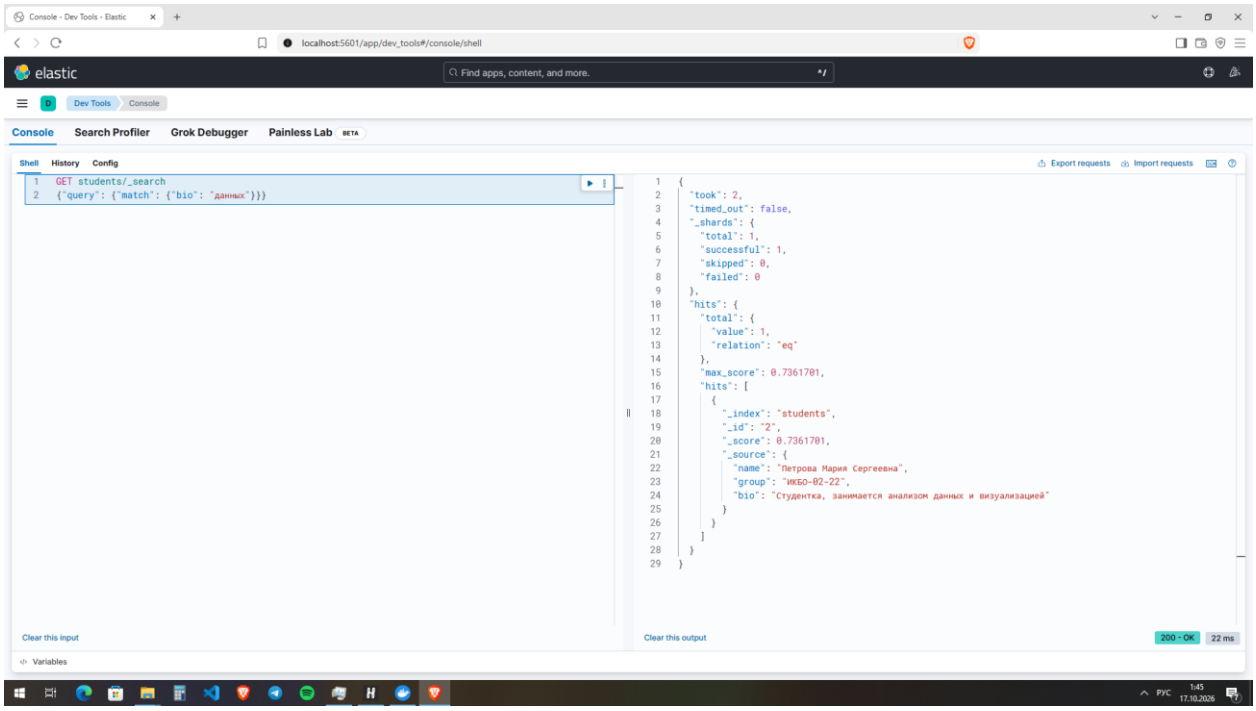



Рисунок 9 — Kibana Dev Tools: полнотекстовый поиск по полю bio

## 5.5. PostgreSQL

В `psql` (рисунок 10) получены версия сервера (16.15) и параметры хранения: каталог данных `/var/lib/postgresql/data/pgdata` (том `practice01_postgres_data`), сборщик журналов `logging_collector =`

on, каталог журналов /var/log/postgresql. Создана и заполнена таблица pr1_students; её содержимое открыто в pgAdmin 4 (рисунок 11).

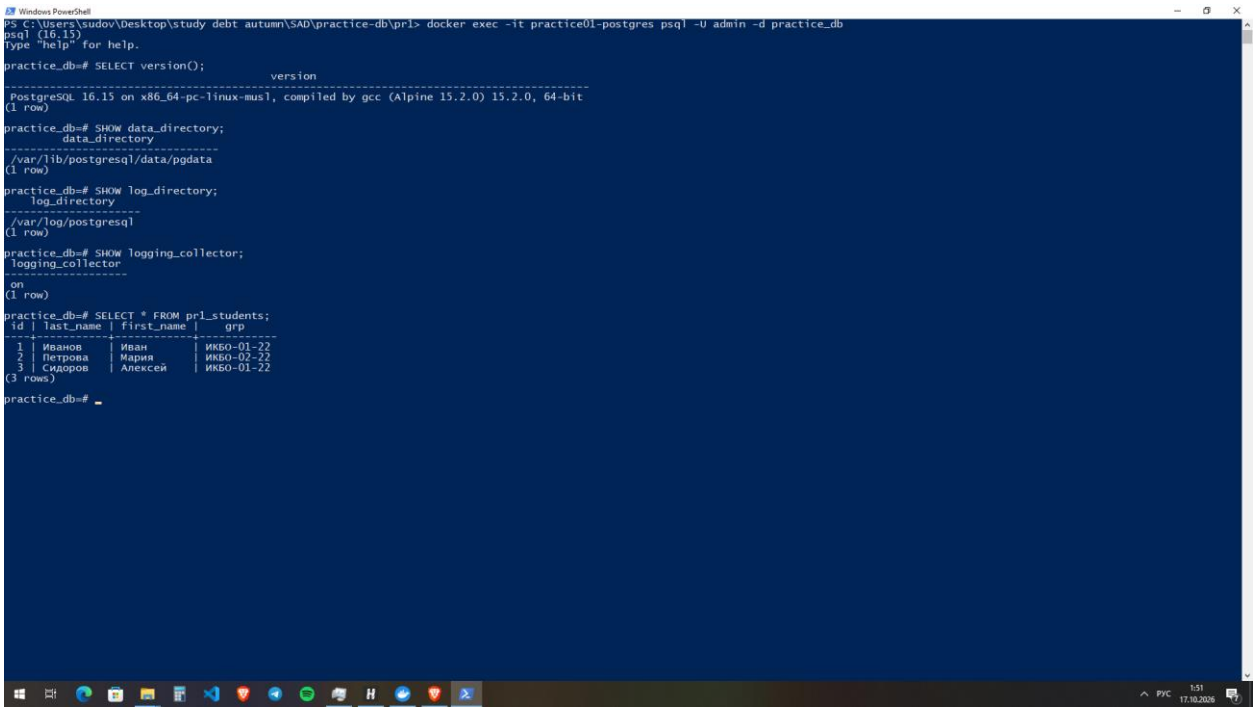



Рисунок 10 — PostgreSQL: версия, каталоги данных и журнала, таблица pr1_students

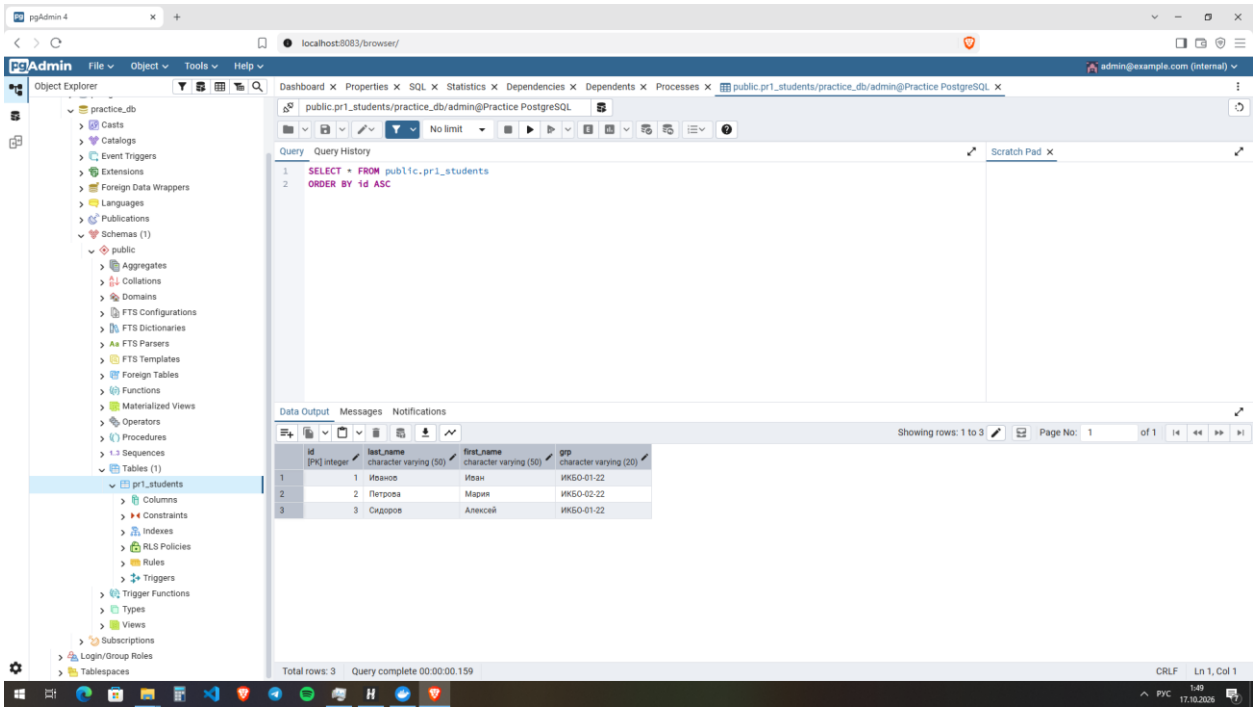



Рисунок 11 — pgAdmin 4: данные таблицы pr1_students

## 6. Места хранения данных и журналов

Данные СУБД хранятся в **именованных томах Docker**: они не зависят от жизненного цикла контейнера и сохраняются после docker compose

down (удаляются только явно, docker compose down -v). Тома размещаются внутри виртуальной машины Docker Desktop (/var/lib/docker/volumes/<том>/_data), где файловая система Linux обеспечивает нужные СУБД права доступа и производительность.

Журналы ведутся на двух уровнях. Первый — файлы самих СУБД: их каталоги смонтированы на хост (bind mount) в pr1\logs\<служба>, поэтому журналы можно открыть обычными средствами Windows и они не пропадают при пересоздании контейнера. Второй — стандартные потоки вывода контейнеров: они собираются драйвером json-file с ротацией 10 МБ × 3 файла (docker compose logs <служба>).

Таблица 3 — Места хранения данных и журналов

| **СУБД** | **Данные: том → каталог** | **Журнал: хост → контейнер** | **Ротация файлов журнала** |
|---|---|---|---|
| Redis | practice01_redis_data → /data (AOF-файлы, dump.rdb) | logs\redis → /var/log/redis/redis.log | встроенной нет; выполняется внешними средствами (logrotate) |
| MongoDB | practice01_mongodb_data → /data/db, practice01_mongodb_config → /data/configdb | logs\mongodb → /var/log/mongodb/mongod.log | по команде db.adminCommand({logRotate: 1}) |
| Neo4j | practice01_neo4j_data → /data | logs\neo4j → /logs (neo4j.log, debug.log, http.log, query.log, security.log) | встроенная (log4j2, по размеру файла) |
| Elasticsearch | practice01_es_data → /usr/share/elasticsearch/data | logs\elasticsearch → /usr/share/elasticsearch/logs (practice-cluster*.log/json, gc.log) | встроенная (log4j2, по дате и размеру) |
| PostgreSQL | practice01_postgres_data → /var/lib/postgresql/data | logs\postgres → /var/log/postgresql/postgresql-ГГГГ-ММ-ДД.log | log_rotation_age = 1d, log_rotation_size = 10MB |

Фактическое подключение томов и каталогов к контейнерам подтверждается командой docker inspect (рисунок 12), каталоги журналов на хосте — рисунком 13. В журнале PostgreSQL (рисунок 14) видны запуск сервера, подключения и выполненные SQL-операторы

(`log_statement = all`, `log_connections = on`); время записей — московское (`log_timezone = Europe/Moscow`).

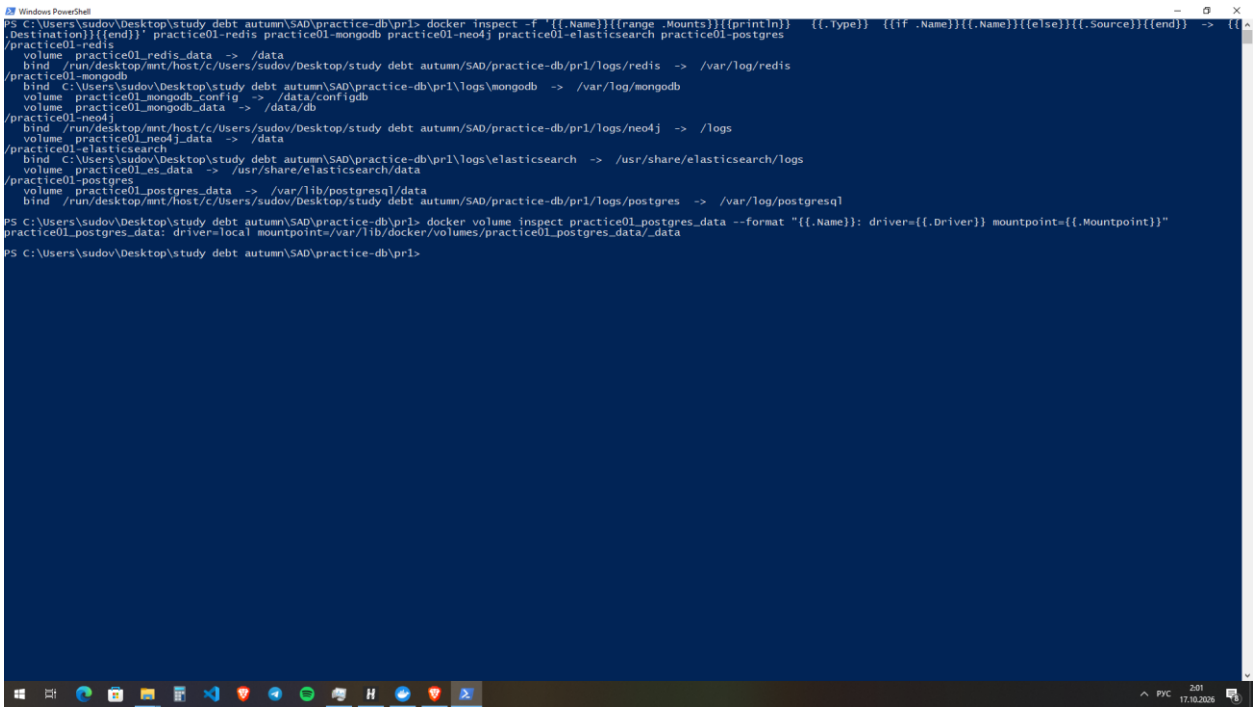



Рисунок 12 — Тома и каталоги, смонтированные в контейнеры СУБД

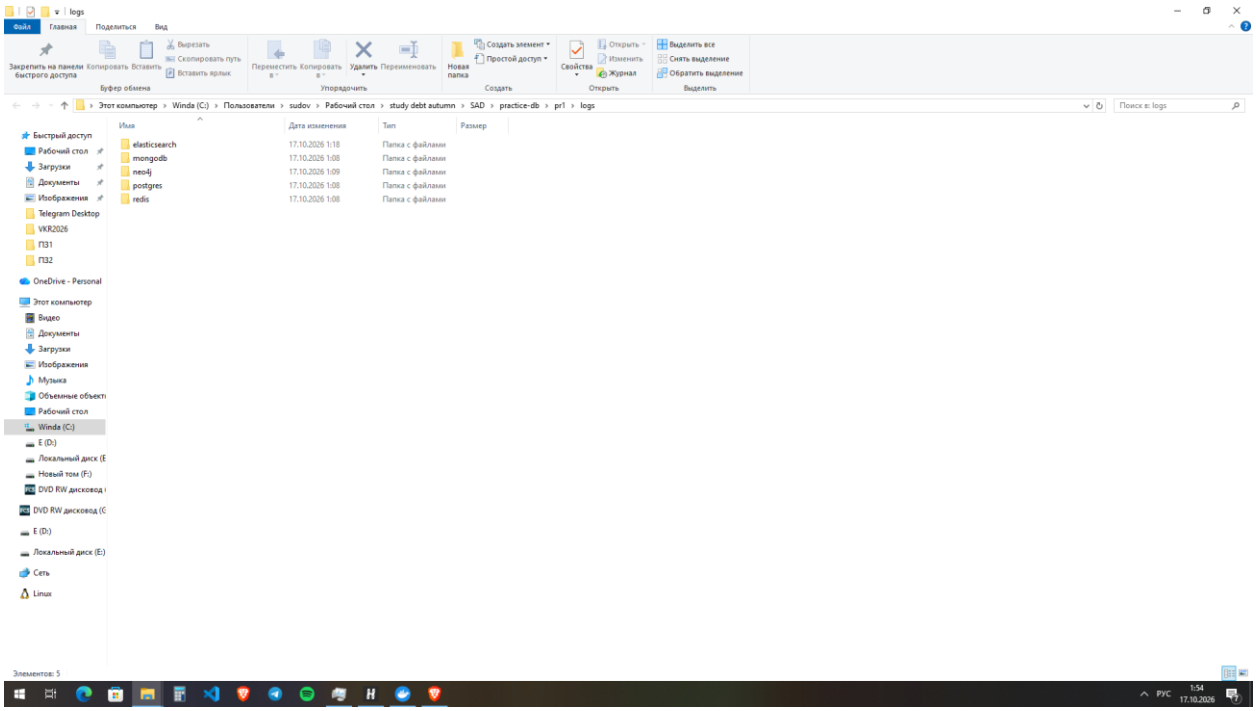



Рисунок 13 — Каталог журналов СУБД на хосте

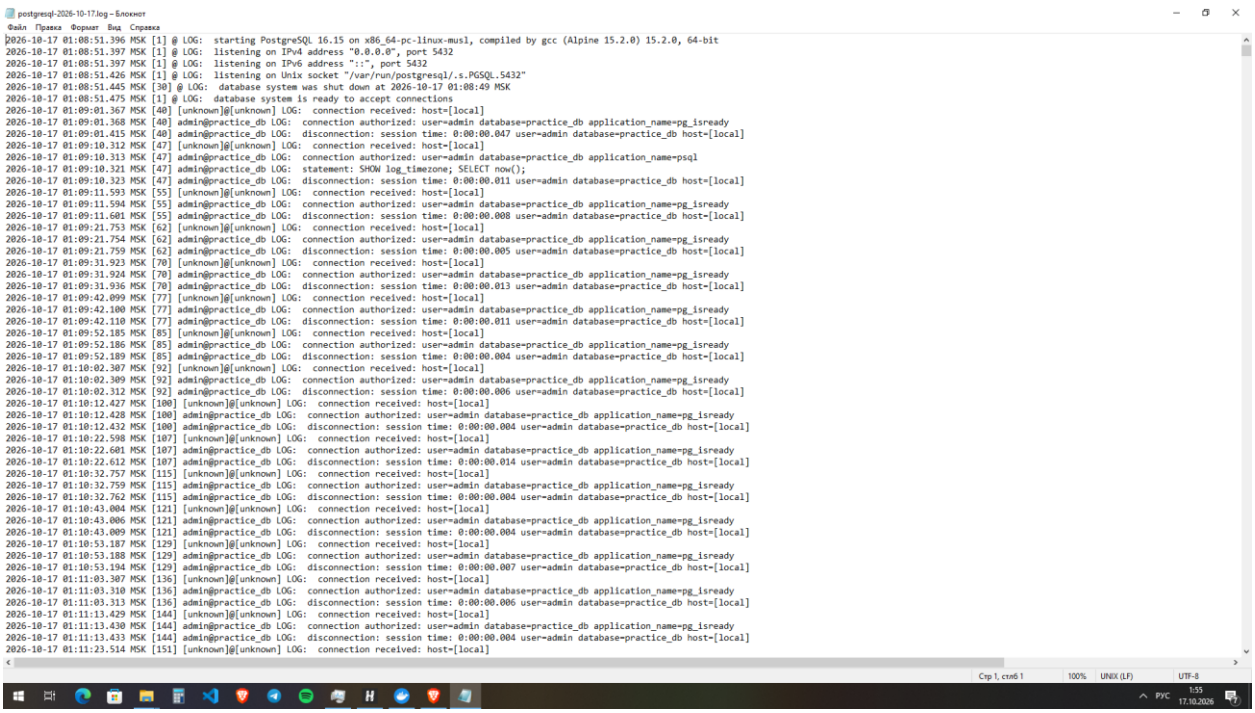


Рисунок 14 — Файл журнала PostgreSQL на хосте

Настройки журналирования подобраны с учётом объёма записей. Проверки готовности выполняются регулярно, поэтому без ограничений журналы росли бы постоянно: для MongoDB включён режим `--quiet` (без записи каждого подключения — у `mongod` нет ротации по размеру), а журнал Neo4j `query.log` остаётся пустым, так как журнал запросов в Community Edition не ведётся. Все проверки разделов 5 и 6 автоматизированы скриптом `check.sh`; его вывод сохранён в каталоге `pr1\evidence`.

## 7. Задания для самостоятельной работы

Методические указания [2] предлагают шесть заданий для самостоятельной работы; все шесть выполнены на развёрнутом стенде (таблица 4). Данные заданий 2–6 записывались в отдельные объекты — контейнер `practice01-postgres-init`, база `selfwork` в MongoDB, ключ `visits:page:/schedule` в Redis, узлы `SNUser` в Neo4j, индекс `books` в Elasticsearch — и после проверки удалены, чтобы не смешиваться с данными практических работ №2 и №3. Файлы заданий лежат в каталоге `pr1\selfwork`, команды всех рисунков раздела — в сценарии `tools\term_pr1.ps1`.

Таблица 4 — Задания для самостоятельной работы

| № | Задание | Как выполнено | Результат | Рис. |
|---|---|---|---|---|
| 1 | Сменить пароли во всех конфигурациях и проверить работу служб | скрипт `rotate_passwords.py`: смена пароля внутри каждой службы, правка `pr1\.env` и `lab1\.env`, пересоздание контейнеров | новые пароли принимаются, прежние отклоняются; веб-интерфейсы и шлюз ЛР №1 работают; пароли методички возвращены | 15, 16 |
| 2 | Скрипт инициализации PostgreSQL через /docker-entrypoint-initdb.d | `initdb\01_schema.sql`, `02_data.sql`, том в `docker-compose.initdb.yml` | 3 таблицы, 2 группы, 5 студентов, 15 отметок; при перезапуске инициализация пропущена | 17 |
| 3 | MongoDB: вложенные документы и агрегация суммы заказов | коллекция `orders`: покупатель — вложенный документ, позиции — массив; `$unwind`, `$group`, `$multiply` | суммы заказов 88 000, 68 200 и 48 000; по покупателям — 136 000 и 68 200 | 18 |
| 4 | Redis: счётчик посещений INCR с TTL | `INCR`, `EXPIRE 10`, `TTL`, ожидание истечения | счётчик 1…6, TTL 10 → 5 → −2, ключ удалён | 19 |
| 5 | Neo4j: «друзья друзей» без существующих друзей | 7 пользователей, 8 связей `FRIEND`, шаблон пути длины 2 с исключением себя и прямых друзей | для Анны: Дина (2 общих друга), Глеб, Егор | 20 |
| 6 | Elasticsearch: полнотекстовый поиск с ранжированием | индекс `books` (10 книг, анализатор `russian`), `multi_match` с весом названия `title^2` | 7 совпадений, упорядочены по оценке релевантности `_score` | 21 |

## 7.1. Смена паролей во всех конфигурациях

Переменные `MONGO_INITDB_ROOT_PASSWORD`, `POSTGRES_PASSWORD`, `NEO4J_AUTH` и `PGADMIN_DEFAULT_PASSWORD` действуют только при первом запуске контейнера на пустом томе. Для уже инициализированных томов одной правки `.env` недостаточно: служба продолжит принимать старый пароль, а клиенты с новым паролем получат отказ. Поэтому скрипт `selfwork\rotate_passwords.py` выполняет смену в три шага:

– меняет пароль внутри работающей службы её собственной командой: `ALTER USER` в PostgreSQL, `changeUserPassword` в MongoDB, `ALTER CURRENT USER SET PASSWORD` в Neo4j, `CONFIG SET requirepass` в Redis, пароль пользователя pgAdmin;

– записывает новые значения в `pr1\.env` (стенд, mongo-express, Redis Commander, проверки готовности) и в `lab1\.env` (строки подключения `PG_DSN` и `REDIS_URL` сервиса ЛР №1);

– пересоздаёт контейнеры с изменённой конфигурацией (`docker compose up -d`) и ждёт статуса healthy; данные остаются в томах.

Затем для каждой службы проверяется вход новым и прежним паролем (рисунок 15): Redis, PostgreSQL (по сети Docker, как подключаются клиенты: внутри контейнера локальный вход доверенный), MongoDB, Neo4j и вход в веб-интерфейс pgAdmin принимают новый пароль и отклоняют прежний; mongo-express отвечает 401 на старый пароль basic auth; Redis Commander переподключился к Redis с новым паролем; шлюз ЛР №1 строит отчёт, читая PostgreSQL и Redis по новым строкам подключения.

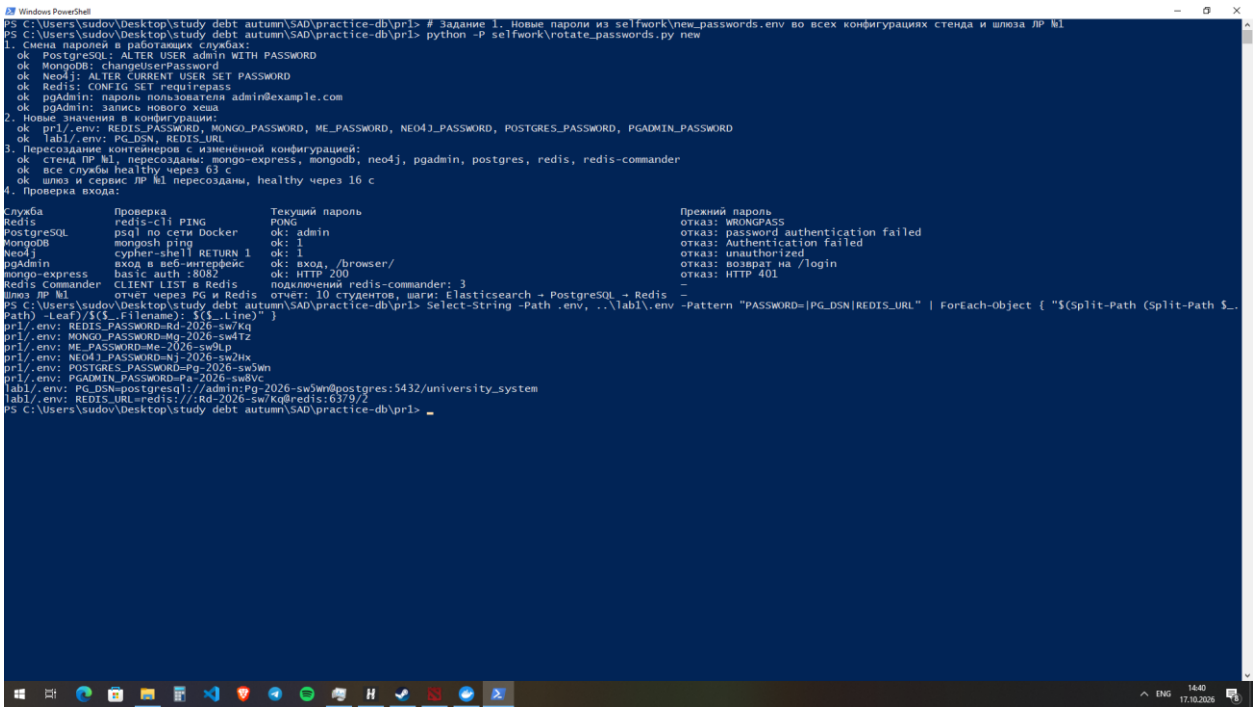


Рисунок 15 — Смена паролей во всех конфигурациях и проверка служб

Тем же скриптом возвращены пароли методических указаний (рисунок 16): на них рассчитаны скрипты практических работ №2–3 и конфигурация ЛР №1. Два наблюдения. Утилита pgAdmin `setup.py update-user` не принимает пароли короче 6 символов, поэтому пароль методички `admin` записывается напрямую через модель пользователя pgAdmin (`selfwork\pgadmin_set_password.py`). В Elasticsearch и Kibana учебной конфигурации безопасность отключена (`xpack.security.enabled=false`), паролей у них нет; в рабочей среде её

включают и задают пароли встроенных пользователей `elastic` и `kibana_system`.

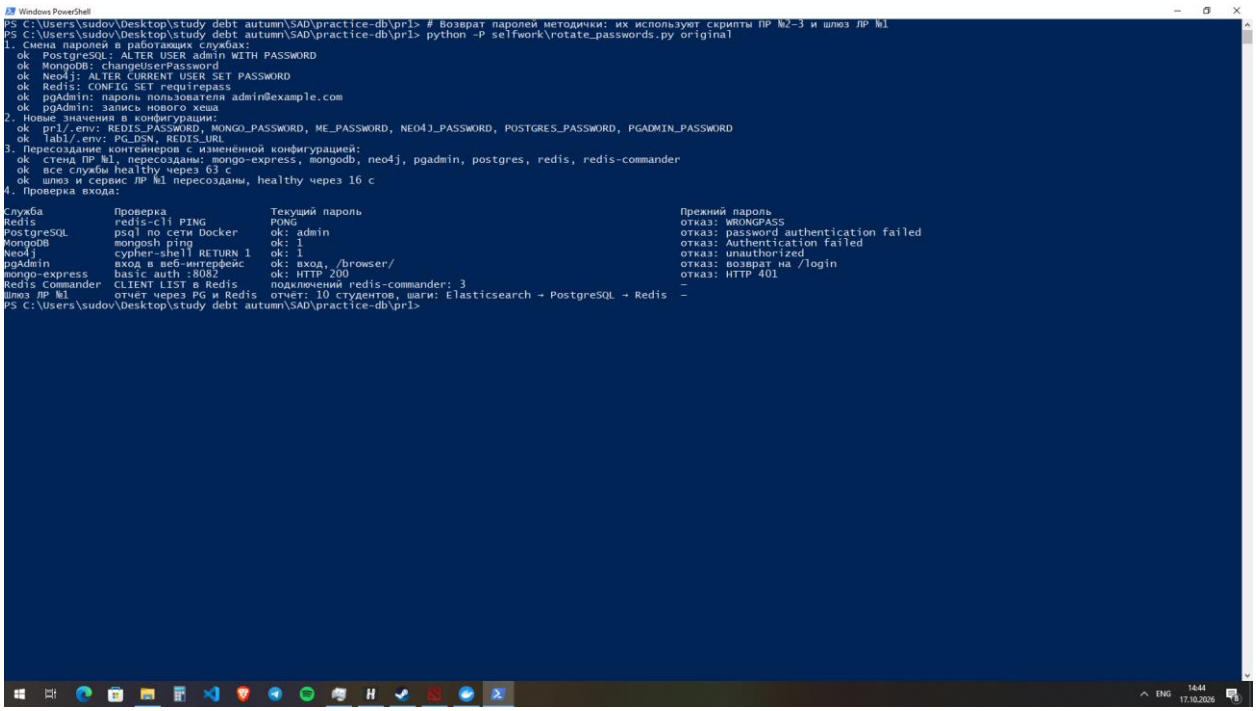



Рисунок 16 — Возврат паролей методических указаний и повторная проверка

## 7.2. Скрипт инициализации PostgreSQL

Официальный образ PostgreSQL при первом запуске, когда каталог данных пуст, выполняет по алфавиту файлы `*.sql` и `*.sh` из `/docker-entrypoint-initdb.d`. Скрипты `selfwork\initdb\01_schema.sql` (таблицы групп, студентов и отметок с внешними ключами) и `02_data.sql` (тестовые данные) подключены туда томом только для чтения. Демонстрация — в отдельном контейнере с новым томом (`selfwork\docker-compose.initdb.yml`, порт 5434): том основного стенда уже инициализирован, и скрипты на нём не выполнились бы. По журналу контейнера (рисунок 17) видно, что оба файла выполнены по порядку, а таблицы и данные созданы; после перезапуска на том же томе образ пишет «Skipping initialization».

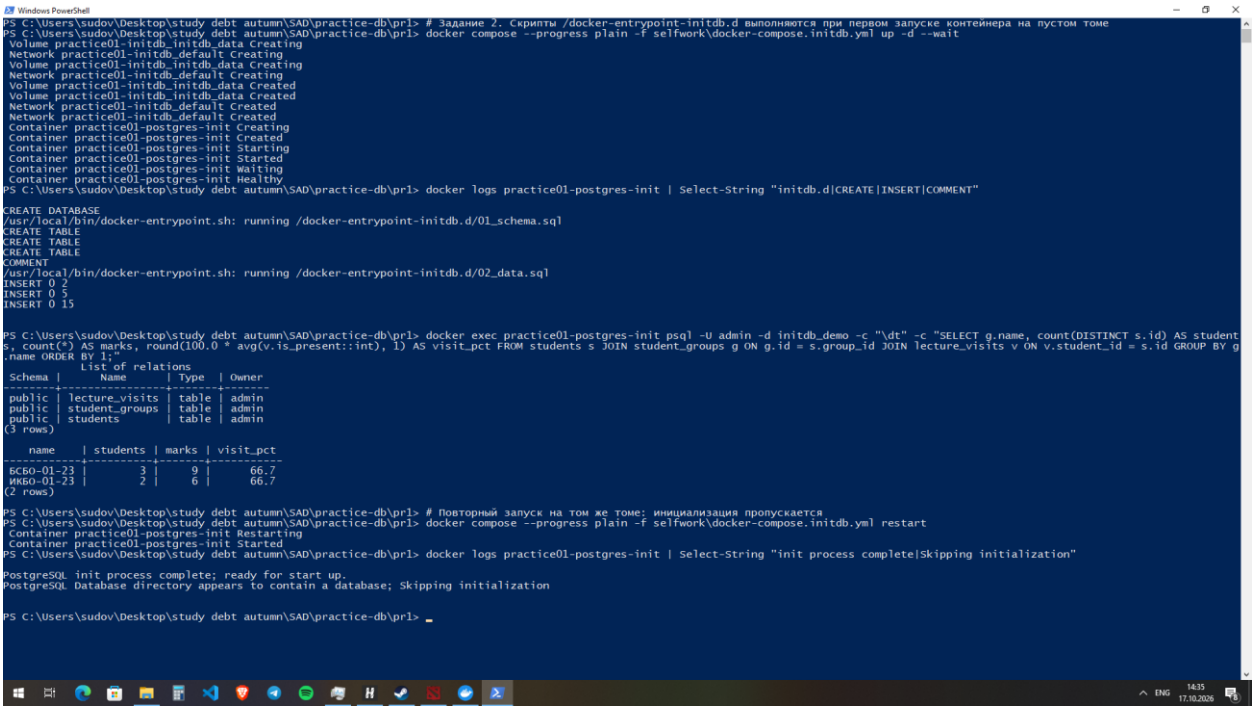


Рисунок 17 — Выполнение скриптов /docker-entrypoint-initdb.d при первом запуске

## 7.3. MongoDB: вложенные документы и агрегация

Коллекция `orders` в базе `selfwork` хранит заказ одним документом: покупатель — вложенный документ, позиции — массив вложенных документов с ценой и количеством (рисунок 18). Сумма заказа считается конвейером агрегации: `$unwind` разворачивает позиции, `$group` складывает произведения количества на цену; второй запрос группирует по покупателю и считает число его заказов.

```
db.orders.aggregate([{$unwind: "$items"},
  {$group: {_id: "$_id", customer: {$first: "$customer.name"}, positions: {$sum: 1},
            total: {$sum: {$multiply: ["$items.qty", "$items.price"]}}}},
  {$sort: {_id: 1}}])
```

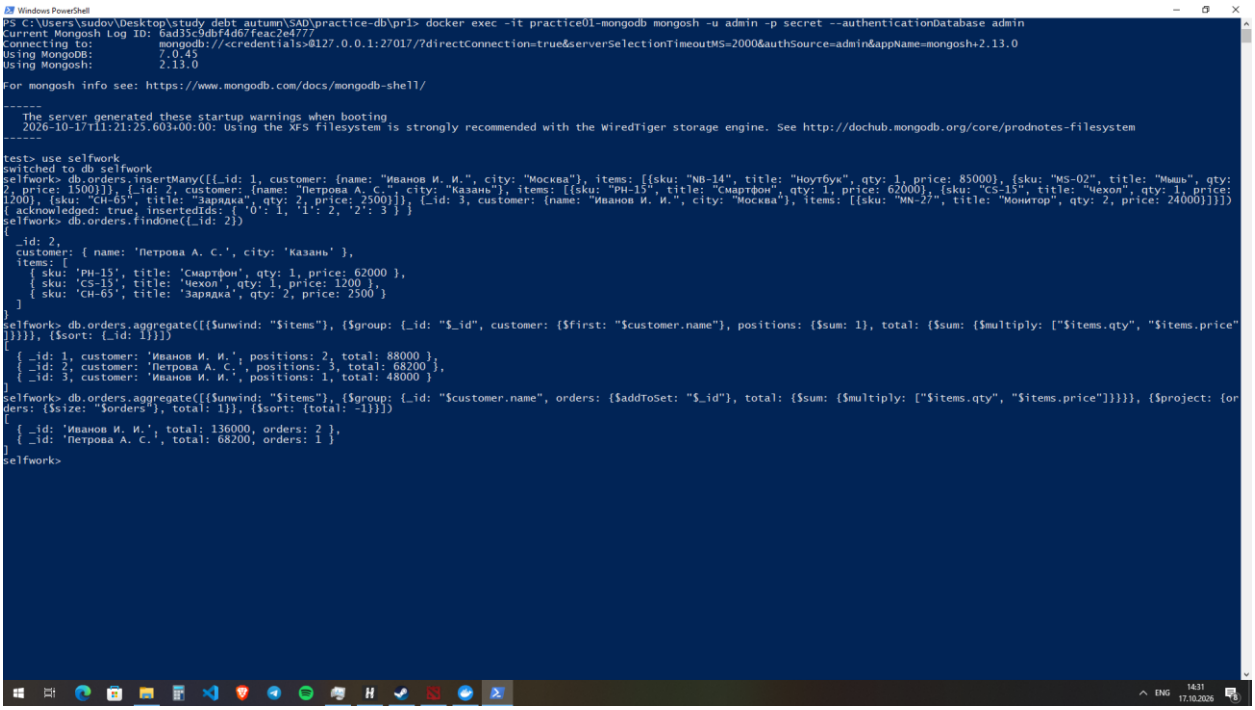



Рисунок 18 — MongoDB: заказы с вложенными позициями и агрегация сумм

## 7.4. Redis: счётчик посещений с TTL

Счётчик посещений страницы — строковый ключ `visits:page:/schedule`, который атомарно увеличивает команда `INCR` (ключ создаётся при первом вызове). `EXPIRE 10` задаёт время жизни, `TTL` показывает остаток. Через 5 секунд счётчик увеличен ещё раз, и TTL при этом не сбросился. После истечения срока `TTL` возвращает −2, `EXISTS` — 0: Redis удалил ключ сам (рисунок 19).

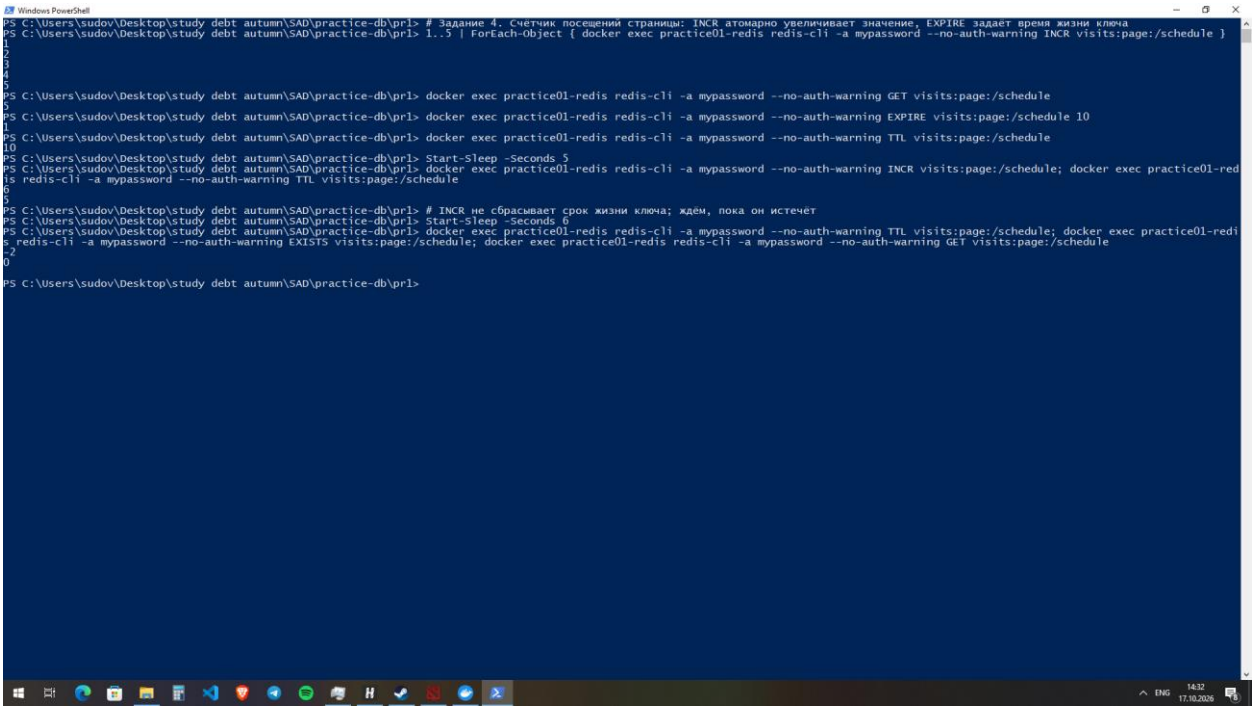



Рисунок 19 — Redis: INCR, EXPIRE и автоматическое удаление ключа

### 7.5. Neo4j: «друзья друзей»

Граф социальной сети — 7 узлов `SNUser` и 8 связей `FRIEND`; связь дружбы взаимна, поэтому в запросах её направление не учитывается. Кандидаты в друзья — узлы на расстоянии двух связей, кроме самого пользователя и его прямых друзей; число общих друзей служит рангом рекомендации (рисунок 20).

```
MATCH (me:SNUser {name: 'Анна'})-[:FRIEND]-(f)-[:FRIEND]-(fof)
WHERE fof <> me AND NOT (me)-[:FRIEND]-(fof)
RETURN fof.name AS candidate, count(DISTINCT f) AS mutual_friends, collect(DISTINCT
    f.name) AS via
ORDER BY mutual_friends DESC, candidate
```

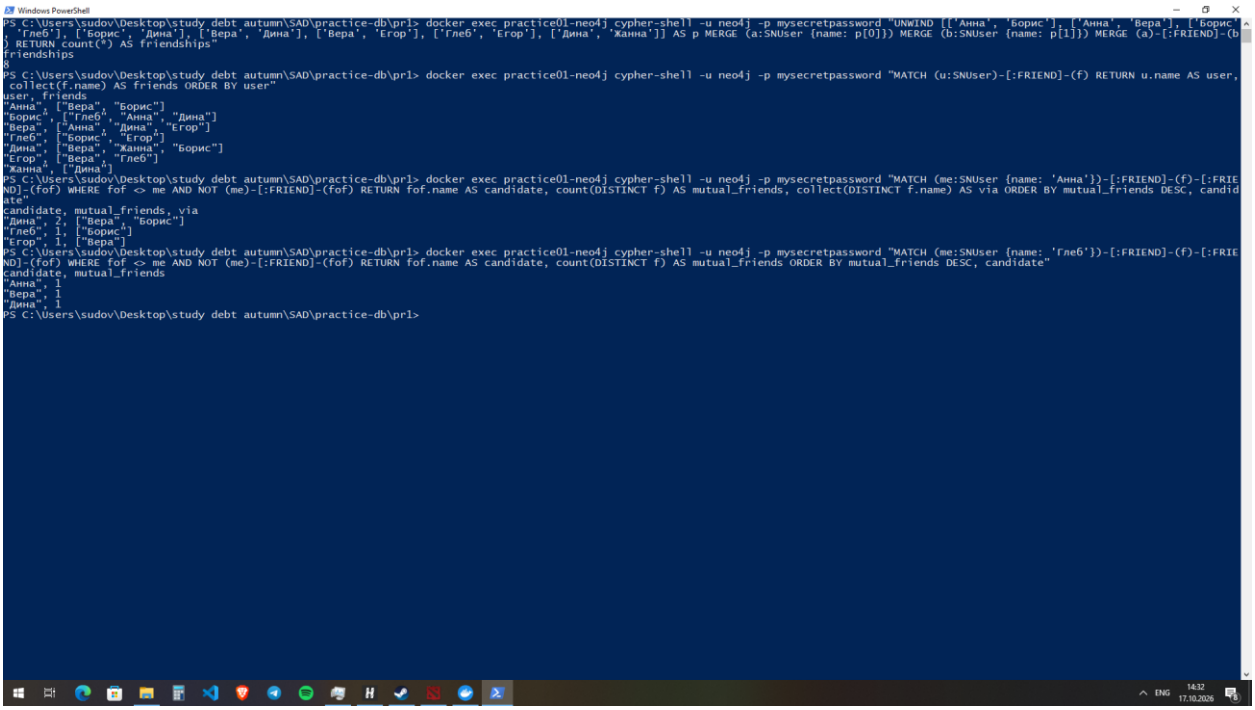



Рисунок 20 — Neo4j: граф дружбы и поиск «друзей друзей»

### 7.6. Elasticsearch: полнотекстовый поиск с ранжированием

Индекс `books` — 10 книг по проектированию систем и данных с полями `title` и `description` (встроенный анализатор `russian`: приведение к нижнему регистру, стоп-слова, стемминг), автором, годом и тегами. Запрос «масштабируемые системы хранения данных» ищет по названию с весом 2 и по описанию; Elasticsearch упорядочивает 7 найденных книг по оценке BM25 `_score`: выше те, где больше слов запроса и они реже встречаются в индексе (рисунок 21). Подсветка показывает, что слова найдены в других формах: «систем», «хранение».

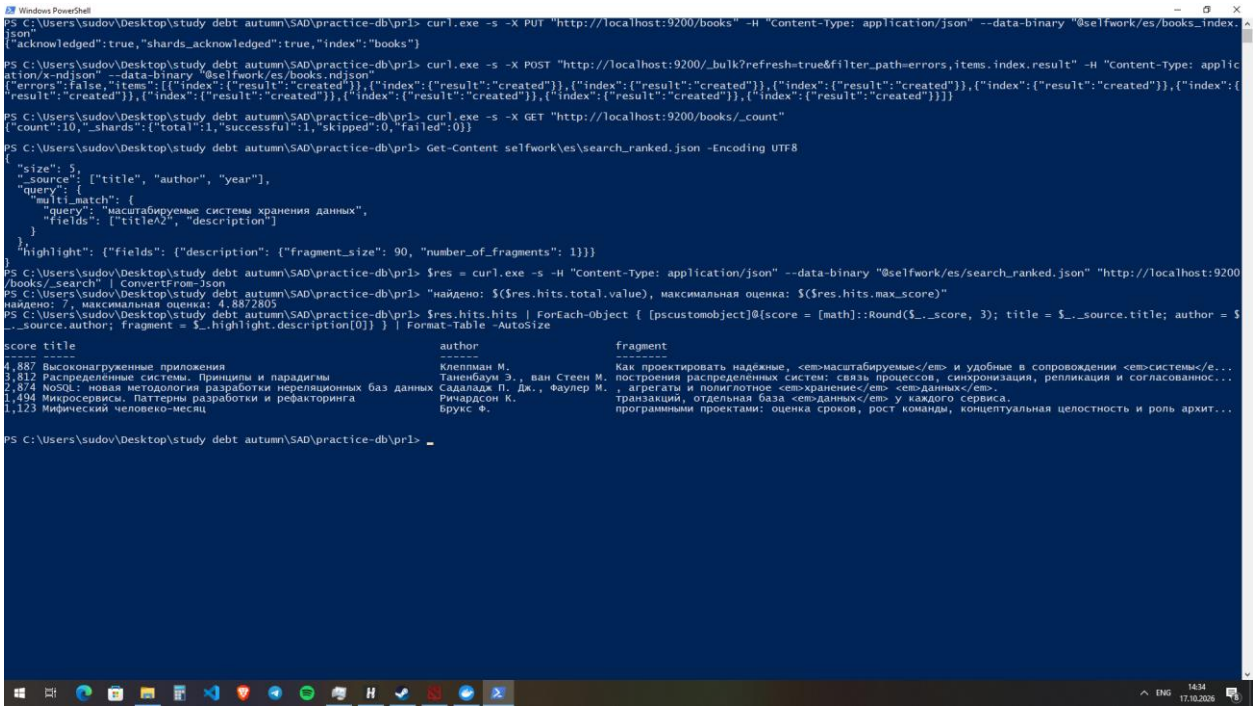


Рисунок 21 — Elasticsearch: индекс books и поиск с ранжированием по релевантности

## 8. Выводы

В ходе работы с помощью Docker Compose развёрнут стенд из пяти СУБД разных моделей данных — Redis, MongoDB, Neo4j, Elasticsearch и PostgreSQL — с веб-интерфейсами к каждой из них. Доступность всех служб подтверждена тремя способами: автоматическими проверками готовности (статус `healthy`), командами клиентов командной строки внутри контейнеров и работой через веб-интерфейсы; в каждой СУБД выполнены базовые операции записи и чтения.

Для каждой службы настроено место хранения данных — именованный том Docker, независимый от контейнера, — и место хранения журналов: файлы журналов СУБД выведены в каталоги на хосте, а потоки вывода контейнеров ограничены ротацией. Отличия от примеров методических указаний (порт 5433 для PostgreSQL, лимиты памяти, способ проверки готовности Neo4j) вызваны условиями конкретного рабочего места и обоснованы в разделе 4. Стенд является основой для следующих практических и лабораторных работ.

Выполнены все шесть заданий для самостоятельной работы: смена паролей во всех конфигурациях с проверкой каждой службы и возвратом исходных значений, автоматическая инициализация PostgreSQL SQL-скриптами, агрегация вложенных документов MongoDB, счётчик Redis с временем жизни ключа, поиск «друзей друзей» в Neo4j и полнотекстовый поиск Elasticsearch с ранжированием по релевантности.

Работа выполнена 17.10.2026; рисунки сняты в тот же день с экрана рабочего компьютера.

**Список использованных источников**

1. Задание на практические и лабораторные работы по дисциплине «Проектирование архитектуры ПО». — Москва, 2026. — Текст : электронный.

2. Практическая работа №1. Развертывание контейнеризированных сред для нескольких типов баз данных : методические указания. — 2026. — 43 с. — Текст : электронный.

3. Docker Compose : документация. — URL: https://docs.docker.com/compose/ (дата обращения: 17.10.2026). — Текст : электронный.

4. Redis Documentation. — URL: https://redis.io/docs/latest/ (дата обращения: 17.10.2026). — Текст : электронный.

5. MongoDB Manual. — URL: https://www.mongodb.com/docs/manual/ (дата обращения: 17.10.2026). — Текст : электронный.

6. Neo4j Cypher Manual. — URL: https://neo4j.com/docs/cypher-manual/current/ (дата обращения: 17.10.2026). — Текст : электронный.

7. Elasticsearch Guide 8.16. — URL: https://www.elastic.co/guide/en/elasticsearch/reference/8.16/index.html (дата обращения: 17.10.2026). — Текст : электронный.

8. PostgreSQL 16 Documentation. — URL: https://www.postgresql.org/docs/16/index.html (дата обращения: 17.10.2026). — Текст : электронный.

9. Садаладж П. Дж., Фаулер М. NoSQL: новая методология разработки нереляционных баз данных / П. Дж. Садаладж, М. Фаулер. — Москва : Вильямс, 2013. — 192 с. — Текст : непосредственный.

# Приложение А. Файл docker-compose.yml

```
# ПР №1. Развёртывание пяти СУБД в Docker: Redis, MongoDB, Neo4j, Elasticsearch, PostgreSQL
# (+ веб-интерфейсы Redis Commander, Mongo Express, Neo4j Browser, Kibana, pgAdmin 4).
#
# Хранение:
#   данные  -> именованные тома Docker (practice01_*_data), переживают удаление контейнеров;
#   логи    -> 1) файлы самих СУБД в каталоге ./logs/<служба> на хосте (bind mount);
#              2) stdout/stderr контейнеров -> драйвер json-file с ротацией 10 МБ x 3 файла.
# Учётные данные берутся из файла .env рядом с этим файлом.


name: practice01


x-logging: &default-logging
  driver: json-file
  options:
    max-size: "10m"
    max-file: "3"


services:
  # ---------------- Redis (key-value) ----------------
  redis:
    image: redis:7-alpine
    container_name: practice01-redis
    restart: unless-stopped
    ports:
      - "6379:6379"
    # --appendonly yes / --save 60 1 -> персистентность AOF + RDB-снимок в /data
    # --logfile -> журнал Redis в /var/log/redis (смонтирован на хост)
    command: >
      redis-server
      --requirepass ${REDIS_PASSWORD}
      --appendonly yes
      --save 60 1
      --maxmemory 256mb
      --maxmemory-policy allkeys-lru
      --loglevel notice
      --logfile /var/log/redis/redis.log
    volumes:
      - redis_data:/data
      - ./logs/redis:/var/log/redis
    healthcheck:
      test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "--no-auth-warning", "ping"]
      interval: 10s
      timeout: 5s
      retries: 3
    mem_limit: 300m
    logging: *default-logging


  redis-commander:
    image: rediscommander/redis-commander:latest
    container_name: practice01-redis-commander
    restart: unless-stopped
    environment:
      # db0 — данные ПР №1, db1 — студенты ПР №2 (ключ = номер зачётной книжки), db2 — кэш ПР №3
      - REDIS_HOSTS=local:redis:6379:0:${REDIS_PASSWORD},pr2:redis:6379:1:${REDIS_PASSWORD},pr3:
          redis:6379:2:${REDIS_PASSWORD}
    ports:
      - "8081:8081"
    depends_on:
      redis:
        condition: service_healthy
    mem_limit: 256m
    logging: *default-logging


  # ---------------- MongoDB (документная) ----------------
  mongodb:
    image: mongo:7
    container_name: practice01-mongodb
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
      MONGO_INITDB_DATABASE: practice_db
    ports:
```

```
    - "27017:27017"
  # --wiredTigerCacheSizeGB 0.5 -> кеш движка не больше 0,5 ГБ
  # --logpath/--logappend      -> журнал mongod в /var/log/mongodb (смонтирован на хост)
  # --quiet                    -> не писать в журнал каждое подключение (иначе healthcheck
  #                               раздувает файл, а ротации по размеру у mongod нет)
  command:
    - mongod
    - --wiredTigerCacheSizeGB
    - "0.5"
    - --logpath
    - /var/log/mongodb/mongod.log
    - --logappend
    - --quiet
  volumes:
    - mongodb_data:/data/db
    - mongodb_config:/data/configdb
    - ./logs/mongodb:/var/log/mongodb
  healthcheck:
    test: ["CMD", "mongosh", "--quiet", "-u", "${MONGO_USER}", "-p", "${MONGO_PASSWORD}",
           "--authenticationDatabase", "admin", "--eval", "db.adminCommand('ping').ok"]
    interval: 30s
    timeout: 5s
    retries: 5
    start_period: 20s
  mem_limit: 1g
  logging: *default-logging


mongo-express:
  image: mongo-express:1-20-alpine3.19
  container_name: practice01-mongo-express
  restart: unless-stopped
  environment:
    ME_CONFIG_MONGODB_ADMINUSERNAME: ${MONGO_USER}
    ME_CONFIG_MONGODB_ADMINPASSWORD: ${MONGO_PASSWORD}
    ME_CONFIG_MONGODB_URL: mongodb://${MONGO_USER}:${MONGO_PASSWORD}@mongodb:27017/
    ME_CONFIG_BASICAUTH_USERNAME: ${ME_USER}
    ME_CONFIG_BASICAUTH_PASSWORD: ${ME_PASSWORD}
  ports:
    - "8082:8081"
  depends_on:
    mongodb:
      condition: service_healthy
  mem_limit: 256m
  logging: *default-logging


# ---------------- Neo4j (графовая) ----------------
neo4j:
  image: neo4j:5-community
  container_name: practice01-neo4j
  restart: unless-stopped
  ports:
    - "7474:7474"   # HTTP, Neo4j Browser
    - "7687:7687"   # Bolt
  environment:
    NEO4J_AUTH: neo4j/${NEO4J_PASSWORD}
    NEO4J_server_memory_heap_initial__size: 256m
    NEO4J_server_memory_heap_max__size: 512m
    NEO4J_server_memory_pagecache_size: 256m
  volumes:
    - neo4j_data:/data
    - ./logs/neo4j:/logs   # debug.log, query.log, http.log, security.log
  # лёгкая HTTP-проверка: cypher-shell при каждом вызове поднимает отдельную JVM и грузит CPU
  healthcheck:
    test: ["CMD-SHELL", "wget --no-verbose --tries=1 --spider http://localhost:7474 || exit
        1"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 40s
  mem_limit: 1200m
  logging: *default-logging


# ---------------- Elasticsearch (полнотекстовый поиск) ----------------
elasticsearch:
  image: elasticsearch:8.16.0
  container_name: practice01-elasticsearch
  restart: unless-stopped
  environment:
```

```
    - node.name=es-node-1
    - cluster.name=practice-cluster
    - discovery.type=single-node
    - xpack.security.enabled=false
    - xpack.security.enrollment.enabled=false
    - xpack.ml.enabled=false
    - ES_JAVA_OPTS=-Xms512m -Xmx512m
    # писать журналы в файлы каталога logs, а не только в консоль
    - ES_LOG_STYLE=file
  ports:
    - "9200:9200"
  volumes:
    - es_data:/usr/share/elasticsearch/data
    - ./logs/elasticsearch:/usr/share/elasticsearch/logs
  ulimits:
    memlock:
      soft: -1
      hard: -1
  healthcheck:
    test: ["CMD-SHELL", "curl -fs http://localhost:9200/_cluster/health || exit 1"]
    interval: 15s
    timeout: 10s
    retries: 10
    start_period: 30s
  # heap 512 МБ + off-heap память JVM: при лимите 1 ГБ из методички контейнер работал у
      самого предела
  mem_limit: 1536m
  logging: *default-logging


kibana:
  image: kibana:8.16.0
  container_name: practice01-kibana
  restart: unless-stopped
  environment:
    - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
  ports:
    - "5601:5601"
  depends_on:
    elasticsearch:
      condition: service_healthy
  mem_limit: 1536m
  logging: *default-logging


# ---------------- PostgreSQL (реляционная) -----------------
postgres:
  image: postgres:16-alpine
  container_name: practice01-postgres
  restart: unless-stopped
  environment:
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    POSTGRES_DB: ${POSTGRES_DB}
    POSTGRES_INITDB_ARGS: "--encoding=UTF-8"
    PGDATA: /var/lib/postgresql/data/pgdata
  ports:
    - "${POSTGRES_HOST_PORT}:5432"
  # logging_collector=on -> журнал сервера пишется в файлы /var/log/postgresql (смонтирован
      на хост)
  command: >
    postgres
    -c shared_buffers=128MB
    -c max_connections=100
    -c logging_collector=on
    -c log_directory=/var/log/postgresql
    -c log_filename=postgresql-%Y-%m-%d.log
    -c log_rotation_age=1d
    -c log_rotation_size=10MB
    -c log_statement=all
    -c log_connections=on
    -c log_disconnections=on
    -c log_line_prefix='%m [%p] %u@%d '
    -c log_timezone=Europe/Moscow
    -c timezone=Europe/Moscow
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./logs/postgres:/var/log/postgresql
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
```

```
      interval: 10s
      timeout: 5s
      retries: 5
    mem_limit: 512m
    logging: *default-logging


  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: practice01-pgadmin
    restart: unless-stopped
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_PASSWORD}
      PGADMIN_CONFIG_ENHANCED_COOKIE_PROTECTION: "False"
    ports:
      - "8083:80"
    volumes:
      - pgadmin_data:/var/lib/pgadmin
      # сервер "Practice PostgreSQL" регистрируется автоматически при первом запуске
      - ./pgadmin/servers.json:/pgadmin4/servers.json:ro
    depends_on:
      postgres:
        condition: service_healthy
    mem_limit: 512m
    logging: *default-logging


volumes:
  redis_data:
  mongodb_data:
  mongodb_config:
  neo4j_data:
  es_data:
  postgres_data:
  pgadmin_data:
```

# Приложение Б. Смена паролей: selfwork\rotate_passwords.py

```
"""ПР №1, самостоятельное задание 1: смена паролей во всех конфигурациях стенда и проверка служб.


  python -P selfwork/rotate_passwords.py new        перейти на пароли из selfwork/new_passwords.env
  python -P selfwork/rotate_passwords.py original   вернуть пароли методички (их используют ПР №2–3 и ЛР №1)
  python -P selfwork/rotate_passwords.py check      проверить текущие пароли без изменений


Смена в работающих службах не теряет данные: пароль меняется командой самой СУБД (ALTER USER,
changeUserPassword, ALTER CURRENT USER, CONFIG SET requirepass, пароль пользователя pgAdmin), затем новые
значения пишутся в pr1/.env и lab1/.env, и docker compose пересоздаёт контейнеры с изменённой конфигурацией.
Переменные *_INITDB_*, NEO4J_AUTH и PGADMIN_DEFAULT_* действуют только при первом запуске на пустом томе,
поэтому одной правки .env для уже инициализированных томов недостаточно.
Проверка: вход новым паролем проходит, старым — отклоняется; веб-интерфейсы и шлюз ЛР №1 работают.
"""
import http.cookiejar
import json
import re
import subprocess
import sys
import time
import urllib.error
import urllib.parse
import urllib.request
from pathlib import Path


PR1 = Path(__file__).resolve().parent.parent
LAB1 = PR1.parent / 'lab1'
ORIGINAL = {'REDIS_PASSWORD': 'mypassword', 'MONGO_PASSWORD': 'secret', 'ME_PASSWORD': 'admin',
            'NEO4J_PASSWORD': 'mysecretpassword', 'POSTGRES_PASSWORD': 'secret', 'PGADMIN_PASSWORD': 'admin'}




def read_env(path):
    return {m.group(1): m.group(2) for m in re.finditer(r'^([A-Z_0-9]+)=(.*)$',
        path.read_text(encoding='utf-8'), re.M)}




def write_env(path, updates):
    text = path.read_text(encoding='utf-8')
    for k, v in updates.items():
```

```
        text = re.sub(rf'^{k}=.*$', f'{k}={v}', text, flags=re.M)
    path.write_text(text, encoding='utf-8')



def sh(*cmd, cwd=None):
    r = subprocess.run(list(cmd), capture_output=True, text=True, encoding='utf-8', errors='replace', cwd=cwd)
    return r.returncode, (r.stdout + r.stderr).strip()



def ex(container, *cmd, workdir=None):
    return sh('docker', 'exec', *(['-w', workdir] if workdir else []), container, *cmd)



def step(name, rc_out):
    rc, out = rc_out
    print(f'  {"ok " if rc == 0 else "ОШИБКА"} {name}' + ('' if rc == 0 else f': {out[-300:]}'))
    if rc != 0:
        sys.exit(1)



# ---------------------------------------------------------------- проверки входа
def chk_redis(pw):
    rc, out = ex('practice01-redis', 'redis-cli', '-a', pw, '--no-auth-warning', 'PING')
    return 'отказ: WRONGPASS' if 'WRONGPASS' in out else (out.splitlines()[-1] if out else str(rc))



def chk_pg(pw):
    # по сети Docker, как подключаются клиенты (внутри контейнера локальный вход без пароля — trust)
    rc, out = sh('docker', 'run', '--rm', '--network', 'practice01_default', 'postgres:16-alpine', 'psql',
                 f'postgresql://admin:{pw}@postgres:5432/university_system', '-tAc', 'SELECT current_user')
    return 'ok: ' + out if rc == 0 else ('отказ: password authentication failed' if 'password authentication'
        in out else out[-60:])



def chk_mongo(pw):
    rc, out = ex('practice01-mongodb', 'mongosh', '--quiet', '-u', 'admin', '-p', pw,
        '--authenticationDatabase',
                 'admin', '--eval', 'db.runCommand({ping: 1}).ok')
    return 'ok: ' + out.splitlines()[-1] if rc == 0 else 'отказ: Authentication failed'



def chk_neo4j(pw):
    rc, out = ex('practice01-neo4j', 'cypher-shell', '-u', 'neo4j', '-p', pw, '--format', 'plain', 'RETURN 1
        AS ok')
    return 'ok: ' + out.splitlines()[-1] if rc == 0 else 'отказ: unauthorized'



class _NoRedirect(urllib.request.HTTPRedirectHandler):
    def redirect_request(self, *a, **k):
        return None



def chk_pgadmin(pw):
    op = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(http.cookiejar.CookieJar()),
        _NoRedirect)
    token = re.search(r'"csrfToken":\s*"([^"]+)"',
        op.open('http://localhost:8083/login').read().decode()).group(1)
    data = urllib.parse.urlencode({'email': 'admin@example.com', 'password': pw, 'csrf_token': token,
                                   'language': 'en', 'internal_button': 'Login'}).encode()
    try:
        op.open(urllib.request.Request('http://localhost:8083/authenticate/login', data=data))
        loc = ''
    except urllib.error.HTTPError as e:
        loc = e.headers.get('Location', '')
    return 'ok: вход, /browser/' if loc.startswith('/browser') else 'отказ: возврат на /login'



def chk_mongo_express(pw):
    rc, out = sh('curl.exe', '-s', '-u', f'admin:{pw}', '-o', 'NUL', '-w', '%{http_code}',
        'http://localhost:8082/db/admin/')
    return {'200': 'ok: HTTP 200', '401': 'отказ: HTTP 401'}.get(out, out)
```

```
def chk_commander():
    rc, out = ex('practice01-redis', 'redis-cli', '-a', read_env(PR1 / '.env')['REDIS_PASSWORD'],
        '--no-auth-warning',
                 'CLIENT', 'LIST')
    return f'подключений redis-commander: {out.count("name=redis-commander")}'



def chk_gateway():
    b = 'http://localhost:8000'
    tok = json.loads(urllib.request.urlopen(urllib.request.Request(b + '/auth/login',
        data=urllib.parse.urlencode(
        {'username': 'teacher', 'password': 'teacher-pass'}).encode()), timeout=20).read())['access_token']
    q = urllib.parse.urlencode({'term': 'Kafka', 'date_from': '2026-02-09', 'date_to': '2026-05-31', 'trace':
        'true'})
    r = json.loads(urllib.request.urlopen(urllib.request.Request(b + '/api/lab1/report?' + q,
                                                                 headers={'Authorization': 'Bearer ' + tok}),
                                                                     timeout=30).read())
    return f'отчёт: {len(r["students"])} студентов, шаги: ' + ' → '.join(t['store'] for t in r['trace'][:3])



def verify(cur, old):
    rows = [('Redis', 'redis-cli PING', chk_redis(cur['REDIS_PASSWORD']), chk_redis(old['REDIS_PASSWORD'])),
            ('PostgreSQL', 'psql по сети Docker', chk_pg(cur['POSTGRES_PASSWORD']),
                chk_pg(old['POSTGRES_PASSWORD'])),
            ('MongoDB', 'mongosh ping', chk_mongo(cur['MONGO_PASSWORD']), chk_mongo(old['MONGO_PASSWORD'])),
            ('Neo4j', 'cypher-shell RETURN 1', chk_neo4j(cur['NEO4J_PASSWORD']),
                chk_neo4j(old['NEO4J_PASSWORD'])),
            ('pgAdmin', 'вход в веб-интерфейс', chk_pgadmin(cur['PGADMIN_PASSWORD']),
                chk_pgadmin(old['PGADMIN_PASSWORD'])),
            ('mongo-express', 'basic auth :8082', chk_mongo_express(cur['ME_PASSWORD']),
                chk_mongo_express(old['ME_PASSWORD'])),
            ('Redis Commander', 'CLIENT LIST в Redis', chk_commander(), '—'),
            ('Шлюз ЛР №1', 'отчёт через PG и Redis', chk_gateway(), '—')]
    if cur == old:
        rows = [(a, b, c, '—') for a, b, c, _ in rows]
    w = [max(len(r[i]) for r in rows + [('Служба', 'Проверка', 'Текущий пароль', 'Прежний пароль')]) for i in
        range(4)]
    print()
    for r in [('Служба', 'Проверка', 'Текущий пароль', 'Прежний пароль')] + rows:
        print('  '.join(str(x).ljust(w[i]) for i, x in enumerate(r)))



def wait_healthy(cwd, timeout=240):
    t0 = time.time()
    while time.time() - t0 < timeout:
        rc, out = sh('docker', 'compose', 'ps', '--format', '{{.Name}} {{.Status}}', cwd=cwd)
        bad = [l for l in out.splitlines() if '(health: starting)' in l or 'unhealthy' in l or 'Restarting'
            in l]
        if rc == 0 and out and not bad:
            return round(time.time() - t0)
        time.sleep(3)
    raise SystemExit(f'не дождался готовности в {cwd}: {bad}')



def rotate(target):
    cur = {k: v for k, v in read_env(PR1 / '.env').items() if k in ORIGINAL}
    if cur == target:
        print('пароли уже такие'); return cur
    print('1. Смена паролей в работающих службах:')
    step('PostgreSQL: ALTER USER admin WITH PASSWORD', ex('practice01-postgres', 'psql', '-U', 'admin', '-d',
        'postgres',
         '-v', 'ON_ERROR_STOP=1', '-c', f"ALTER USER admin WITH PASSWORD '{target['POSTGRES_PASSWORD']}'"))
    step('MongoDB: changeUserPassword', ex('practice01-mongodb', 'mongosh', '--quiet', '-u', 'admin', '-p',
         cur['MONGO_PASSWORD'], '--authenticationDatabase', 'admin', '--eval',
         f"db.getSiblingDB('admin').changeUserPassword('admin', '{target['MONGO_PASSWORD']}')"))
    step('Neo4j: ALTER CURRENT USER SET PASSWORD', ex('practice01-neo4j', 'cypher-shell', '-u', 'neo4j', '-p',
         cur['NEO4J_PASSWORD'], '-d', 'system',
         f"ALTER CURRENT USER SET PASSWORD FROM '{cur['NEO4J_PASSWORD']}' TO '{target['NEO4J_PASSWORD']}'"))
    step('Redis: CONFIG SET requirepass', ex('practice01-redis', 'redis-cli', '-a', cur['REDIS_PASSWORD'],
         '--no-auth-warning', 'CONFIG', 'SET', 'requirepass', target['REDIS_PASSWORD']))
    step('pgAdmin: пароль пользователя admin@example.com',
         sh('docker', 'cp', str(Path(__file__).with_name('pgadmin_set_password.py')),
            'practice01-pgadmin:/tmp/pgadmin_set_password.py'))
    step('pgAdmin: запись нового хеша', ex('practice01-pgadmin', '/venv/bin/python3',
        '/tmp/pgadmin_set_password.py',
         'admin@example.com', target['PGADMIN_PASSWORD'], workdir='/pgadmin4'))
    print('2. Новые значения в конфигурации:')
    write_env(PR1 / '.env', target)
    print('  ok  pr1/.env: ' + ', '.join(target))
```

```
    write_env(LAB1 / '.env', {
        'PG_DSN': f"postgresql://admin:{target['POSTGRES_PASSWORD']}@postgres:5432/university_system",
        'REDIS_URL': f"redis://:{target['REDIS_PASSWORD']}@redis:6379/2"})
    print('  ok  lab1/.env: PG_DSN, REDIS_URL')
    print('3. Пересоздание контейнеров с изменённой конфигурацией:')
    rc, out = sh('docker', 'compose', 'up', '-d', cwd=PR1)
    changed = sorted({m for m in re.findall(r'Container (\S+)\s+Recreate', out)})
    print('  ok  стенд ПР №1, пересозданы: ' + ', '.join(c.replace('practice01-', '') for c in changed))
    print(f'  ok  все службы healthy через {wait_healthy(PR1)} c')
    sh('docker', 'compose', 'up', '-d', cwd=LAB1)
    print(f'  ok  шлюз и сервис ЛР №1 пересозданы, healthy через {wait_healthy(LAB1)} c')
    return cur



if __name__ == '__main__':
    mode = sys.argv[1] if len(sys.argv) > 1 else 'check'
    if mode == 'check':
        cur = {k: v for k, v in read_env(PR1 / '.env').items() if k in ORIGINAL}
        verify(cur, cur)
    else:
        target = ORIGINAL if mode == 'original' else {k: v for k, v in read_env(Path(__file__).with_name(
            'new_passwords.env')).items() if k in ORIGINAL}
        old = rotate(target)
        print('4. Проверка входа:')
        verify(target, old)
```

# Приложение В. Скрипты инициализации PostgreSQL: selfwork\initdb

```
-- ПР №1, самостоятельное задание 2. Скрипты каталога /docker-entrypoint-initdb.d выполняются
    официальным
-- образом PostgreSQL по алфавиту и только при первом запуске контейнера, пока том данных пуст.
CREATE TABLE student_groups (
    id         serial PRIMARY KEY,
    name       varchar(20)  NOT NULL UNIQUE,
    specialty  varchar(100) NOT NULL
);


CREATE TABLE students (
    id           serial PRIMARY KEY,
    group_id     integer      NOT NULL REFERENCES student_groups (id),
    card_number  varchar(10)  NOT NULL UNIQUE,
    full_name    varchar(150) NOT NULL,
    email        varchar(100)
);


CREATE TABLE lecture_visits (
    student_id  integer NOT NULL REFERENCES students (id),
    visit_date  date    NOT NULL,
    is_present  boolean NOT NULL,
    PRIMARY KEY (student_id, visit_date)
);


COMMENT ON TABLE lecture_visits IS 'Отметки посещения лекций (тестовые данные скрипта
    инициализации)';
-- ПР №1, самостоятельное задание 2: тестовые данные (выполняется после 01_schema.sql).
INSERT INTO student_groups (name, specialty) VALUES
    ('БСБО-01-23', '38.03.05 Бизнес-информатика'),
    ('ИКБО-01-23', '09.03.04 Программная инженерия');


INSERT INTO students (group_id, card_number, full_name, email) VALUES
    (1, '23Б0101', 'Андреев Илья Сергеевич',     '23b0101@edu.mirea.ru'),
    (1, '23Б0102', 'Беляев Даниил Олегович',     '23b0102@edu.mirea.ru'),
    (1, '23Б0103', 'Васильева Анна Игоревна',    '23b0103@edu.mirea.ru'),
    (2, '23Б0301', 'Богданов Алексей Олегович',  '23b0301@edu.mirea.ru'),
    (2, '23Б0302', 'Григорьева Мария Павловна',  '23b0302@edu.mirea.ru');


-- по одной отметке на каждую из трёх недель февраля, посещение — детерминированно по номеру
    студента
INSERT INTO lecture_visits (student_id, visit_date, is_present)
SELECT s.id, d::date, (s.id + extract(day FROM d)::int) % 3 <> 0
FROM students s, generate_series(date '2026-02-09', date '2026-02-23', interval '7 days') AS d;
# ПР №1, самостоятельное задание 2: PostgreSQL со скриптами инициализации из ./initdb.
# Отдельный экземпляр с пустым томом: у основного стенда том уже инициализирован, и скрипты
# /docker-entrypoint-initdb.d на нём не выполнятся.
#   docker compose -f selfwork/docker-compose.initdb.yml up -d --wait
#   docker compose -f selfwork/docker-compose.initdb.yml down -v      (удалить вместе с томом)
```

```
name: practice01-initdb

services:
  postgres-init:
    image: postgres:16-alpine
    container_name: practice01-postgres-init
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: initdb_demo
    ports:
      - "5434:5432"
    volumes:
      - ./initdb:/docker-entrypoint-initdb.d:ro
      - initdb_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin -d initdb_demo"]
      interval: 5s
      timeout: 5s
      retries: 10
    mem_limit: 256m

volumes:
  initdb_data:
```